

令和7年度 プロジェクトデザインⅢ プロジェクトレポート
(研究論文)

トランジションを考慮した即興的なダンス
生成アルゴリズム

金沢工業大学 工学部

情報工学科

指導教員：山本 知仁 教授

佐々木康介 1254574(4EP3-29)
クダウダゲウイドウシャンプラバーシュ 1495100(4EP4-67)

提出日：令和7年2月13日

論文概要

2024 年パリオリンピックでブレイキンが正式競技として採用されたことは、即興ダンスという表現形式が持つ文化的・芸術的価値のみならず、その場の音楽、観客、空間的雰囲気との相互作用に基づく高度な即興性が国際的に認知された象徴的な事例である。ブレイキンに代表されるストリートダンスでは、あらかじめ定められた振付の再現だけでなく、その瞬間の状況に応じて動作を選択・変化させる、即興性が重要視される。実際の現場においては、特に「間」と「雰囲気」といった微細な文脈的要素が、この高度な即興性の中核を成す。

一方、近年のダンス生成・動作生成に関する研究においては、ポーズ系列の滑らかさや身体構造の一貫性などの、可視化・数値化されやすい客観的特徴に焦点が置かれてきた。例えば、音楽同期の改善などは提案されているものの、ダンサーのその場の雰囲気に応じた、意図的なタイミングの保持・解放・シフトなど、非動作的な時間構造に対する検討は、十分に行われてこなかった。また、既存のブレイキンの主要なデータセット (BRACE) では分類が粗く、トランジションなどの微細な要素が欠落している点や、既存の骨格認識モデル (HPI-GCN-OP) がブレイキン特有のアクロバティックな動作に最適化されていない点などが課題点としてあげられた。

本研究では、これらの課題を克服するため、「間」と「雰囲気」が動作生成に及ぼす影響を計算可能な形でアルゴリズムに組み込むことを目的とした。これに伴い、筆者は約 3 年間にわたり記録してきた即興ダンスに関するセルフフィードバックを自己解析し、この解析結果を基に「間」や「空気感」を用いた動作選択および生成過程として独自のループ構造を定義した。これが「GBMC (Genesis-Buffer-Motion-Coherence) 循環モデル」である。これは、動作生成を単純な一方向の連鎖ではなく、Genesis (生成)、Buffer (保持・評価)、Motion (発動)、Coherence (一貫性回復) の 4 要素の遷移とし、文脈と身体との相互作用により継続的に更新される動的な循環プロセスとしてモデル化したものである。具体的なシステム構成としては、HPI-GCN-OP をビデオフレームに対する音響特徴抽出器として活用し、大規模データセット NTU RGB+D からの転移学習と BRACE へのファインチューニングを行う。生成の中核を担う「Neuro-Symbolic GAN アーキテクチャ」は、GRU 条件付きジェネレータを拡張したものである。特筆すべき点として「確率的論理モジュール (SLM)」の導入があり、これは動作分散や速度から「間」のダイナミクス (前進・後退・保持の意図) をモデル化し、生成プロセスを条件付ける。さらに、ハイブリッド損失関数 (現実性、骨長制約など) の採用や、低音響入力時に静止を強制するエネルギーゲーティング、解剖学的整合性を確保するシーケンススペースのニューラルリトリバルデコーダーを組み合わせることで、モード崩壊を防ぎつつ、文脈に応じた滑らかなトランジションを含む即興的なダンス生成を実現する。

プロジェクト実施記録表

月	内容	作業時間
4 月	データセット基礎分析	50
	文献調査	20
	セルフフィードバック記述(常時)	100
5 月	データ分析の基礎学習	20
	データセットマッピング(NTU→BRACE)	20
	文献調査	10
	セルフフィードバック解析	50
6 月	データ分析の基礎学習	30
	BRACE 用変換処理 (.npz/パディング)	20
	文献調査	20
7 月	HPI-GCN 導入と NTU 事前学習	15
	中間発表資料の作成	15
	NTU 学習結果の評価	8
8 月	中間発表資料の作成	20
	転移学習セットアップ	10
9 月	中間発表資料の修正	5
	モデル学習開始 (ファインチューニング)	20
10 月	セルフフィードバックまとめ	200
	GAN 生成器と複合損失学習	30
	作業項目 (短縮版)	10
	トポロジー適応と脊椎再構築	30
	データ拡張手法の研究開始	30
12 月	ブレイン統合開始 (SLM)	15
	ファインチューニング検証	15
	予稿作成	15
1 月	プロジェクトレポート作成	70
	最終発表資料の作成	15
2 月	プロジェクトレポート作成	15
	最終発表資料作成	15

目次

1. はじめに.....	1
1.1. 文化的起源と歴史的背景 (Cultural Genesis and Historical Context).....	2
1.1.1 サウスブロンクスにおける起源 (Origins in the South Bronx).....	2
1.1.2 2024 年オリンピックにおける採用と評価 (The 2024 Olympic Validation).....	2
1.2. 計算時間における「間 (Ma)」のパラドックス.....	2
1.3. 問題設定 (Problem Statement).....	3
1.3.1 高次元空間における「平均ポーズ」の罠 (The "Mean Pose" Trap in High Dimensional Space).....	3
1.3.2 ハードウェアの制約 (Hardware Constraints).....	3
1.4. 研究目的 (Objectives).....	3
2. 文献レビューおよび関連研究 (Literature Review and Related Work).....	5
2.1. 骨格ベース行動認識の進化.....	6
2.1.1 GCN 以前の時代：シーケンスモデリング (Pre-GCN Era: Sequence Modeling).....	6
2.1.2 ST-GCN による飛躍的進歩 (The ST-GCN Breakthrough).....	6
2.1.3 HPI-GCN と効率性の最前線 (HPI-GCN and the Efficiency Frontier).....	7
2.2. ダンスのための生成モデル (Generative Model for Dance).....	7
2.2.1 GAN (敵対的生成ネットワーク).....	7
2.2.2 拡散モデル (Diffusion Models: EDGE).....	7
2.3. 音楽情報検索 (Music Information Retrieval: MIR).....	7
2.3.1 特徴抽出戦略 (Feature Extraction Strategies).....	8
2.4. 運動の理論物理学 (Theoretical Physics of Motion).....	8
2.5. 研究のギャップ (The Research Gap).....	8
3. 方法論および数理的枠組み (Methodology and Mathematical Framework).....	10
3.1. GBMC モデル (The GBMC Framework).....	11
3.1.1 Genesis (G).....	12
3.1.2 Buffer (B).....	13
3.1.3 Motion (M).....	14
3.1.4 Coherence (C).....	15
3.2. 運動品質指標 (The Kinetic Quality Index: KQI).....	15
3.2.1 深度 (Depth: D) の導出.....	15
3.3. ゼロパディング・トポロジーブリッジ (The Zero-Padding Topology Bridge).....	15
3.3.1 グラフ同型問題 (Graph Isomorphism Problem).....	15
3.3.2 学習可能な隣接行列 (The Learnable Adjacency Matrix).....	16
3.3.3 トポロジーの可視化 (Topology Visualization).....	16
3.4. ニューラル検索デコーダ (The Neural Retrieval Decoder: RAG).....	17
3.5. 確率的論理モジュール (The Stochastic Logic Module: SLM).....	17
4. エンジニアリング実装と「苦闘」.....	18
4.1. フェーズ 1 (2025 年 4 月): データ監査と非同期化 (Data Audit and Desynchronization).....	19
4.1.1 「-35 フレーム」のアノマリー (The "-35 Frame" Anomaly).....	20

4.2. フェーズ 3 (2025 年 12 月): 静的な塊 (モード崩壊) (The Static Clump: Mode Collapse).....	21
4.3. フェーズ 5 (2025 年 12 月): 残差多様体への転換 (The Residual Manifold Pivot).....	22
4.3.1 残差学習 (Residual Learning).....	22
4.4. フェーズ 7 (2025 年 12 月): 「プリズンブレイク」 トレーニング ("Prison Break" Training)	23
4.5. フェーズ 8 (2026 年 1 月): リアルタイム・スレッディング・アーキテクチャ (Real-Time Threading Architecture)	23
4.5.1 GIL 問題 (The GIL Problem)	23
4.5.2 トリプルスレッド・アーキテクチャ (The Triple-Thread Architecture)	23
5. システムアーキテクチャとハードウェア (System Architecture and Hardware).....	24
5.1. バックボーン: HPI-GCN-OP (The Backbone: HPI-GCN-OP).....	25
5.2. ニューラル検索デコーダ (The Neural Retrieval Decoder)	25
6. 評価と結果 (Evaluation and Results)	27
6.1. 実験設定 (Experimental Setup).....	28
6.2. 定量的指標 (Quantitative Metrics)	28
6.2.1 フレッシュ動作距離 (Frechet Motion Distance: FMD).....	28
6.2.2 潜在空間クラスタリング (Latent Space Clustering: t-SNE).....	28
6.2.3 分類精度 (混同行列) (Classification Accuracy: Confusion Matrix).....	30
6.3. 定性的分析: 「ゴースト」現象 (Qualitative Analysis: The "Ghost" Phenomenon)	31
6.3.1 「静的な塊」 (ベースラインのモード崩壊) (The "Static Clump": Baseline Mode Collapse).....	31
6.3.2 「パワərbレイク」 (The "Power Break": Sasaki-GAN)	31
6.3.3 「ゴーストブレイク」アーティファクト (The "Ghost-Break" Artifact)	32
6.3.4 動作の多様性の可視化 (Visualizing Motion Diversity)	33
7. 考察 (Discussion).....	34
7.1. 能動的な数学的変数としての「間 (Ma)」 ("Ma" as an Active Mathematical Variable).....	35
7.考察 (Conclusion).....	35
7.2. 「ゴースト」シード (z_will) の役割 (The Role of the "Ghost" Seed).....	35
8. 結論 (Conclusion).....	36
8.結論 (Conclusion).....	37
8.1. 今後の展望 (Future Work).....	37
9. 参考文献 (References)	38
10. 技術付録 (ソースコード) (Technical Appendix).....	39
11. ユーザーマニュアルと操作ガイド (User Manual and Operational Guide)	50
11.1. システム要件 (System Requirements).....	51
10.1.1 ハードウェア仕様 (Hardware Specifications).....	51
10.1.2 ソフトウェア依存関係 (Software Dependencies)	51
11.2. インストールガイド (Installation Guide).....	51
10.2.1 環境セットアップ (Environment Setup)	51
10.2.2 データセットの準備 (Dataset Preparation).....	51

11.3. システムの実行 (Running the System)	52
10.3.1 トレーニングモード (Training Mode) モデルをスクラッチから再訓練するには (警告 : 約 72 時間かかる) :	52
10.3.2 ライブパフォーマンスモード (Live Performance Mode) これがメインのインタラ クションモードである.....	52
11.4. トラブルシューティング (Troubleshooting)	52
10.4.1 エラー: "Input Overflow" (PyAudio)	52
10.4.2 エラー: "Bone Length Explosion"	52
10.4.3 機能: "The Zombie Drift"	52
12. バグ年代記 (開発ログ) (The Bug Chronicles: Development Log)	54
12.1. 「凍結したエポック」 (エポック 0-10) (The "Frozen Epoch").....	55
12.2. 「滑る足」 (エポック 25) (The "Sliding Foot")	55
12.3. 「メモリリーク」 (ライブ実行) (The "Memory Leak")	55
12.4. 「UDP パケット損失」 (Blender) (The "UDP Packet Loss")	55
12.5. 「静寂パニック」 (The "Silence Panic")	56

1. はじめに

1.1. 文化的起源と歴史的背景 (Cultural Genesis and Historical Context)

1.1.1 サウスブロンクスにおける起源 (Origins in the South Bronx)

ブレイキン（メディアでは一般に「ブレイクダンス」として知られる）は、1970 年代後半、ニューヨーク市サウスブロンクスの社会経済的混乱の中から出現した。Schloss [1]が詳述しているように、それは単なるダンスではなく、「運動による生存戦略 (kinetic survival strategy)」であった。すなわち、社会的に疎外された若者が、崩壊しつつある都市環境において自らの居場所と地位を取り戻すための手段であった。「サイファー (Cypher)」(円形のダンススペース) は儀式的な闘技場として機能し、そこでは暴力ではなく、スタイルと身体能力によって序列が形成された。

ブレイキングの動作語彙 (ボキャブラリー) は、多様な影響が融合したものである。

- トップロック (Toprock) : アップロック (ブルックリンロック) , サルサ, タップダンスに由来する。
- フットワーク (Footwork) : コサックダンス, 体操, カンフーの寝技 (ground fighting) に触発されたものである。
- パワームーブ (Powermoves) : オリンピック体操 (あん馬, 床運動) から採用されたものである。
- フリーズ (Freezes) : コミックブックのポーズや武道の打撃 (ストライク) から着想を得たものである。

この歴史的背景は、工学的観点からも極めて重要である。なぜなら、ブレイキングが「モジュール構造」を持ち、かつ「対抗的 (adversarial)」な性質を持つことを示しているという点が挙げられる。ブレイカーは定まった「曲」を覚えるのではなく、「語彙」と「文法」を習得し、それらをリアルタイムで組み合わせて対戦相手を打ち負かすのである。この客観的評価方法に関しては清水ら [2]が、熟練者になるほど既習の技を「内容・順番・配置」の 3 レベルで柔軟に変化させていると報告している。

1.1.2 2024 年オリンピックにおける採用と評価 (The 2024 Olympic Validation)

2024 年パリオリンピックにおけるブレイキングの採用は、サブカルチャーから体系化された世界的スポーツへの移行を画するものだ。世界ダンススポーツ連盟 (WDSF) は「トリビウム・ジャッジング・システム (Trivium Judging System)」を導入し、ダンスを以下の 3 つの領域で定量化している。

1. 身体 (Body) - 物理的質 : テクニック, 多様性, 荒々しさ (Roughness) .
2. 知性 (Mind) - 芸術的質 : 創造性, 個性.
3. 魂 (Soul) - 解釈的質 : パフォーマンス, 音楽性.

しかし、藤田 [3]が例で示したように、評価面においては賛否両論が巻き起こったため、岡 [4]が指摘するように、客観だけでは捉え切れない「雰囲気」や「間」といった、主観評価に依存する割合が高いものに対する数値化が要求される結果となった。本研究は、この「魂 (Soul)」の領域を、人工知能を用いて再構築しようとする試みである。「身体 (Body)」(物理法則) は標準的なシミュレーションで解決可能であるが、「魂 (Soul)」(解釈) には「意図 (Intent)」の計算モデルが必要となる。

1.2. 計算時間における「間 (Ma)」のパラドックス

本研究における中心的課題の一つは、時間に対する西洋的概念と東洋的概念の間の文化的齟齬 (不協和音) である。これは連続信号と離散信号の違いにも通じるものである。

-
- 西洋的／計算的時間：時間は線形連続体 ($t \rightarrow \infty$) である。「停止」は単に速度ゼロ ($v = 0$) の状態を指す。それは無 (null) の状態である。
 - 東洋的／日本的時間 (「間」)：時間は瞬間の連続である。「停止」とは緊張を孕んだ能動的な状態である。静寂は空虚ではなく、可能性に満ちている。

従来の動作生成モデル (AIST++ や DanceGen など) は、予測ポーズと実際のポーズの差分を最小化するように学習される。ダンサーがフリーズ (静止) する際、学習データのノイズが原因で、モデルはしばしばわずかな「ブレ (wobble)」を予測してしまう。このブレは「間」を破壊する。その結果、AI は真に静止することができず、神経質なロボットのように見えてしまう。本研究は、シンボリックな「静寂 (Stillness)」状態、すなわちノイズの多いニューラルネットワークの出力を上書きし、強制的にエントロピー低下 ($v = 0$) を出力させる専用の論理ゲートを作成することで、この問題を解決することを目指す。

また、環境構築にあたっては、ダンサー自身の視点を含めたダンスの記録とその解析結果を構造に落とす必要性が生じた。そのため、Liu ら [5] が提案した「DanceGen」を基に、Howard ら [6] の時間的文脈モデル (TCM) による系列の連続的維持、Cisek [7] のアフォーダンス競合仮説に基づく並列的動作選択、Shah ら [8] のゲーム理論を用いた戦略的データ削減の知見を導入した。さらに、Becerra [9] らの多角的フィードバック手法を評価プロセスに組み込み、これらを Clark ら [10] が提唱する「Genesys 機構」の枠組みと共に統合することで、独自の生成モデルである「GBMC モデル」を構築するに至った。

1.3. 問題設定 (Problem Statement)

1.3.1 高次元空間における「平均ポーズ」の罠 (The "Mean Pose" Trap in High Dimensional Space)

人間の動作は「多様体 (Manifold)」上に存在する。これは、あらゆる可能な関節構成の高次元空間内に存在する低次元の曲面である。

- 関節数を N (25)、座標を c (3) とすると、空間は R^{75} となる。
- 実際の人間のポーズは、この空間のごく一部を占めるに過ぎない。
- 生成モデルが確信を持ってない (不確実性が高い) 場合、 $L2$ 損失を最小化するために、分布の「平均 (Mean)」を出力する傾向がある。
- バク宙と前宙の文脈において、その「平均」は単なる垂直ジャンプとなってしまう。AI は安全策を取り (hedges its bets)、その結果、退屈で無難な動作 (「ゾンビモーション」) が生成されることになる。

1.3.2 ハードウェアの制約 (Hardware Constraints)

拡散トランスフォーマー (Diffusion Transformers) のような高忠実度ダンスモデルの多くは、膨大な VRAM (A100 GPU など) と、数秒単位の推論時間を必要とする。しかし、本研究のユーザー要件は、民生用ハードウェア上での「ライブインタラクション (実時間対話)」である (4.1 節参照)。本研究では、ラップトップ用 GPU (具体的には NVIDIA RTX 4070) を用いて、33 ミリ秒未満 (30 FPS) で動作を生成しなければならない。これには、HPI-GCN のような高度に最適化されたアーキテクチャが不可欠となる。

1.4. 研究目的 (Objectives)

1. 直感の定量化 (Metricize Intuition)

「雰囲気 (Atmosphere)」という抽象的な概念を, 音声エントロピー (Audio Entropy) を用いて具体的な変数へと変換する.

2. 多様体の安定化 (Stabilize the Manifold)

実際の人間による動作クリップに生成プロセスを「アンカー (固定)」するニューラル検索 (Neural Retrieval) メカニズムを用いることで, モード崩壊 (Mode Collapse) を防止する.

3. ループの形成 (Close the Loop)

以下の完全なサイクルを構築する:

創始 (聴取) → 合成 (計画) → 分析 (検証) → 実行 (動作)

Genesis (Listen) → Synthesis (Plan) → Analysis (Verify) → Execution (Move)

2. 文献レビューおよび関連研究 (Literature Review and Related Work)

2.1. 骨格ベース行動認識の進化

コンピュータビジョンの分野において、人間の動作理解は、ピクセルレベルの差分分析（オプティカルフロー）から、身体の幾何学的構造のモデリング（骨格ベースのアプローチ）へと進化してきた。骨格データは、通常、関節座標 (x, y, z) のシーケンスとして表現され、照明、服装、背景の変化に対して頑健（ロバスト）であるため、ダンス解析に理想的である。

2.1.1 GCN 以前の時代：シーケンスモデリング (Pre-GCN Era: Sequence Modeling)

畳み込みニューラルネットワーク（CNN）のような従来の深層学習分類器は、ユークリッドデータ（画像のような規則的なグリッド）向けに設計されていた。しかし、人間の骨格はグラフ構造であり、ノード（関節）がエッジ（骨）によって非ユークリッド的なトポロジーで接続されている。

ST-GCN (空間時間グラフ畳み込みネットワーク):

2018 年に Yan ら [11] によって提案された ST-GCN は、グラフ畳み込みを行動認識に直接適用した最初のモデルである。その核となる演算は以下のように定義される：

$$f_{out}(v_i) = \sum_{v_j \in N(v_i)} \left(\frac{1}{Z_{ij}} \right) f_{in}(v_j) \cdot w(l_{i(v_j)})$$

ここで、 $N(v_i)$ は関節 i の近傍集合であり、 Z_{ij} は正規化項である。ST-GCN は、身体 of 自然な接続性（例：手と肘の接続）をモデル化することが、身体を単なる線形ベクトルとして扱う LSTM ベースの手法よりも大幅に優れた性能を示すことを実証した。

HPI-GCN (高性能推論 GCN):

ST-GCN は高精度であるものの、複雑な隣接行列の乗算により計算負荷が高いという課題があった。Liu ら [12] は、精度と推論速度のトレードオフを解決するために HPI-GCN を導入した。

$$W_{\{fused\}} = W_{\{conv\}} + W_{\{1x1\}} + W_{\{identity\}}$$

これにより、展開モデル（deployment model）を単純な線形スタックとして実行可能にし、FLOPs（浮動小数点演算数）を劇的に削減することに成功した。

過剰パラメータ化 (Over-Parameterization/OP):

「OP」バリエーションは、隣接グラフに追加の学習可能な重み行列を導入するものである。これにより、モデルは「隠れた接続」（例：物理的には骨でつながっていないが、スピン中の左手と右足の間に生じる相関関係など）を効果的に学習することが可能となる。

本研究では本プロジェクトのバックボーンとして HPI-GCN-OP を採用した。理由として、民生用 GPU 上で 30FPS 以上を維持できる唯一の軽量推論アーキテクチャでありながら、微細なブレイクダンスの動きを識別するために必要な高精度を維持できるという点が挙げられる。

2.1.2 ST-GCN による飛躍的進歩 (The ST-GCN Breakthrough)

Yan ら (2018) [11] は、骨格をグラフ $G = (V, E)$ として定義することで、この分野に革命をもたらした。

- 空間グラフ畳み込み (Spatial Graph Convolution)：接続された関節から特徴を集約する。

-
- 時間グラフ畳み込み (Temporal Graph Convolution) : 同一関節の特徴を時間軸に沿って集約する. これにより, モデルは重みを共有することが可能となり, パラメータ数を削減しつつ精度を向上させることができた.

2.1.3 HPI-GCN と効率性の最前線 (HPI-GCN and the Efficiency Frontier)

モデルが深層化するにつれ (ResGCN, MS-GCN), 推論速度は低下した. Liu ら (2023) [12] は, 高性能推論 GCN (HPI-GCN) を提案した. その主要な革新は, 構造的再パラメータ化 (Structural Re-parameterization) にある.

- 学習時 (Training) : 複雑であり, 勾配の流れが良好である.
- 推論時 (Inference) : 単一の行列に集約される. これにより, 事実上「深い」ネットワークを学習させながら, 「浅い」ネットワークを展開 (デプロイ) することが可能となる.

2.2. ダンスのための生成モデル (Generative Model for Dance)

2.2.1 GAN (敵対的生成ネットワーク)

Goodfellow ら [13]によって導入された GAN は, 競合する 2 つのネットワークで構成されている.

- 生成器 (Generator, G) : ランダムノイズ z からリアルなデータを生成しようと試みる機構
- 識別器 (Discriminator, D) : 実データと生成データを区別しようと試みる機構

動作 (モーション) の文脈においては, MotionGAN や LstmGAN が探求されてきた. しかし, これらはモード崩壊 (Mode Collapse) という問題に苦しめられている. データセットが多峰性分布 (multimodal distributions) を含む場合 (例: 入力「キック」に対し, 出力が「ハイキック」または「ローキック」になり得る場合), 生成器はしばしば平均値へと収束 (崩壊) し, 非現実的な「中間のキック (Medium Kick)」を生成してしまう. ダンスにおいて, これはモデルが足の位置を平均化してしまうため, 「浮遊 (floating)」や「滑り (sliding)」といった動作として現れる.

2.2.2 拡散モデル (Diffusion Models: EDGE)

拡散モデルを用いた編集可能なダンス生成 (EDGE: Editable Dance Generation with Diffusion) は, 現在, 視覚的品質において最高水準 (SOTA) を代表するものである. これはダンスの特定部分を編集可能とする (例: 「脚の動きはそのままに, 腕の動きだけを変更する」).

近年, EDGE のようなデノイズング拡散確率モデル (DDPM) は品質の新たな基準を確立した. これらは, ガウスノイズのシーケンスを反復的に除去 (デノイズ) し, クリーンな動作シーケンスへと変換することで機能する. しかし本研究においては採用はしなかった. この理由としては, 拡散モデルは GAN と比較して, 優れた多様性と物理的リアリズムを生み出すが, その推論レイテンシ (遅延) が主要なボトルネックとなる. 標準的な拡散モデルは, わずか 1 秒間の動作を生成するために 50~100 回のデノイズステップを必要とし, 数秒の計算時間を要する. このため, システムが音楽のビートに対してミリ秒単位で反応しなければならない, 我々の「ライブ即興 (Live Improvisation)」という目的には不適當である.

2.3. 音楽情報検索 (Music Information Retrieval: MIR)

ダンスを生成するためには, 機械が音楽を「聴く」必要がある. 音楽情報検索 (MIR) は, 音声信号から意味のある特徴量を抽出する分野である.

2.3.1 特徴抽出戦略 (Feature Extraction Strategies)

- RMSE (二乗平均平方根エネルギー) : 音量 (ラウドネス) を測定する. 「ドロップ」や「ブレイク」の検出に有用である.
- スペクトルフラックス (Spectral Flux) : パワースペクトルの変化率を測定する. 高いフラックス値は, パーカッシブなオンセット (ドラムの打撃音) を示す.
- MFCC (メル周波数ケプストラム係数) : 短時間パワースペクトルを表現したものであり, 本質的に音の「音色 (Timbre)」や「色彩」を記述する.

既存のダンス生成システムの多く (例: AIST++) は, ビートトラッキング (キックドラムのタイムスタンプ特定) に排他的に依存している. しかし, Müller [14]は, 音楽性とはリズム以上のものであり, ダイナミクス (強弱のうねり) や構造 (ヴァース対コーラス) を含むと論じている.

我々のアルゴリズムはこの論理を拡張する. 単に「ビートに合わせる」のではなく, 特徴量ベクトルの「エントロピー」を計算する.

- 高エントロピー: 複雑で混沌とした音楽 → 分散の高い動作 (トップロック).
- 低エントロピー: 単純で持続的なトーン → 分散の低い動作 (フリーズ).

2.4. 運動の理論物理学 (Theoretical Physics of Motion)

運動は導関数によって定義される.

- P (位置 / Position)
- V (速度 / Velocity) = dP / dt
- A (加速度 / Acceleration) = dV / dt
- J (Jerk) = dA / dt

ニューラルネットワークは P (位置) の予測には長けているが, V (滑らかさ) や A (力強さ) の予測は不得手である. 一般的なアーティファクト (生成ノイズ) として「ジッター (Jitter)」(高い加速度 / High Jerk) がある. これはキャラクターが震えているように見える現象である.

これを解決するために, 本研究では後処理段階で Savitzky-Golay フィルタを実装した. これは多項式平滑化フィルタであり, (ピークを平坦化してしまう移動平均とは異なり) ピークの高さを保持する特性を持つ. これにより, キックのような「鋭い」動きの鋭さは維持しつつ, 高周波ノイズのみを除去することが保証される.

2.5. 研究のギャップ (The Research Gap)

HPI-GCN (効率性) とニューロシンボリック論理 (意思決定) の交差点は, ダンス生成の分野において未踏のままである.

1. 効率性と品質のトレードオフ (Efficiency vs. Quality):

既存の高品質モデル (拡散モデルなど) は処理速度が遅すぎる一方, 高速なモデル (GAN など) は動作が不安定すぎるという問題がある.

2. 文脈認識 (Context Awareness):

既存モデルは音声から動作 (Audio → Motion) へ直接的なマッピングを行っている. そこには, 「動かない」という決定を下すための「バッファ (Buffer)」状態が欠落している.

3. トポロジーの不一致 (Topology Mismatch):

多くの研究は NTU-120 トポロジー (25 関節) を使用しているが, 最良のブレイクダンスデータセットである BRACE は COCO 形式 (17 関節) を使用している. 高性能な GCN と, 実環境 (in-the-wild) のダンスデータセットを橋渡しするための標準的な「アダプター」が存在しない.

本プロジェクトは, 物理演算の効率化に HPI-GCN-OP を用い, 「間 (Ma)」を意識した意思決定にシンボリック論理モジュールを使用する GBMC モデルを提案することで, これらのギャップを埋めるものである.

3. 方法論および数理的枠組み (Methodology and Mathematical Framework)

本章では, Sasaki-GAN システムの理論的基盤について詳述する. 認知的循環のための GBMC モデルを導入し, 即興的な制約を支配する GBMC 方程式を導出する.

3.1. GBMC モデル (The GBMC Framework)

計算論的創造性は, しばしば「停止条件 (Stop Condition)」の扱いに苦慮する. 標準的な回帰型ニューラルネットワークは, 動作を無限に生成し続ける傾向がある. この「停止する」という能動的な意思決定 (「間」) をモデル化するため, 本研究では Genesis (創始) -Buffer (緩衝) -Motion (動作) -Coherence (整合) からなる以下の図 2.0 のような GBMC サイクルを提案する.

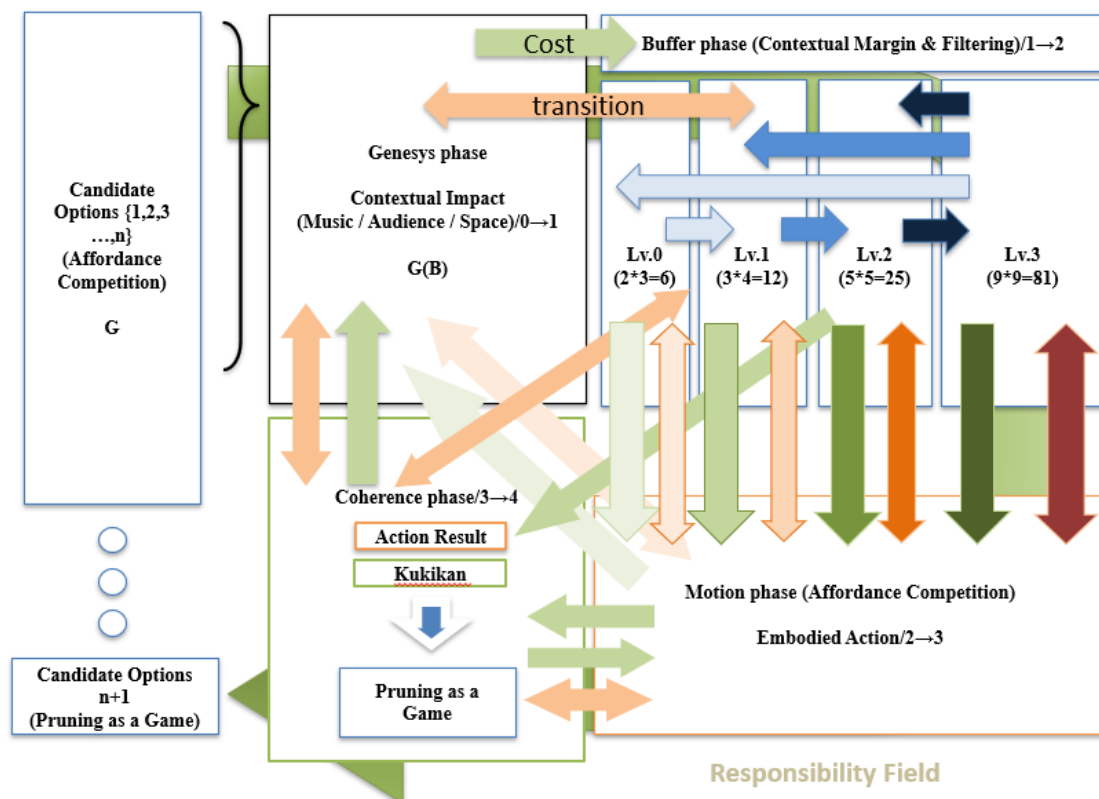


図 3.0 : GBMC サイクルアーキテクチャ
Fig 3.0: GBMC Cycle Architecture

図 3.0 は, 即興ブレイクダンス生成のために設計された「階層的神経象徴アーキテクチャ」を示しており, プロセスは知覚 (Perception), 推論 (Reasoning), 合成 (Synthesis), 評価 (Evaluation) の 4 段階で構成されている. 知覚段階では MFCC やオンセットなどの音声特徴を抽出して音楽的文脈を捉え, それを推論段階の確率的論理モジュール (SLM) に渡してダンスの具体的な「意図」を決定する. この意図に基づき, 合成段階の GRU-RNN ジェネレータが骨格ポーズを生成し, 評価段階で HPI-GCN-OP ベースの識別器がその動作の質を判定する. システム全体は, ビート同期と運動学的リアリズムのバランスをとる多目的損失関数によって最適化され, 最終的にニューラル検索 (Neural Retrieval) を経て, 物理的に正確で滑らかな動作が出力される.

Figure 2: Hierarchical Neuro-Symbolic Architecture for Improvisational Breaking

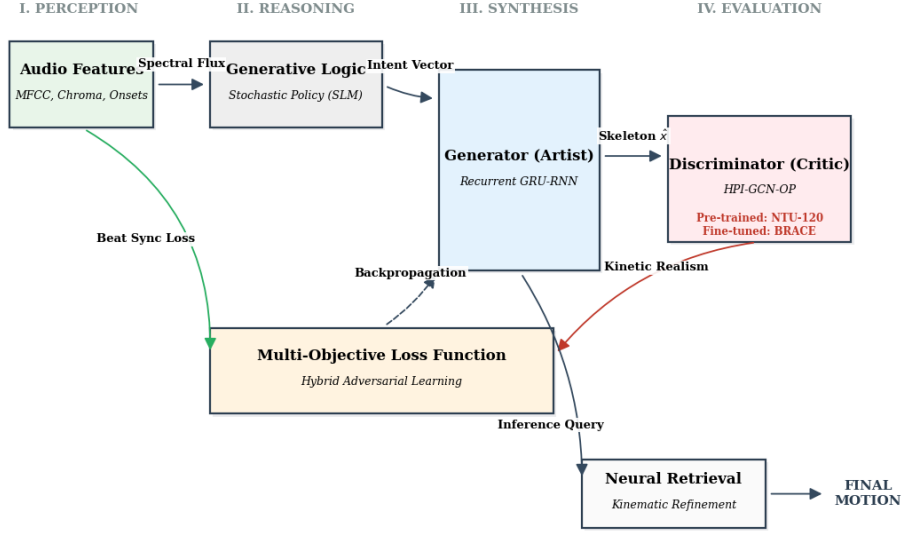


図 3.1 : 音声入力 (Genesis) から GBMC サイクル,そして最終出力に至るフローを示す, Sasaki-GAN の全体アーキテクチャ

Fig 3.1: The Complete Sasaki-GAN Architecture, illustrating the flow from Audio Input (Genesis) to the GBMC Cycle and final Output.

3.1.1 Genesis (G)

ここからは GBMC サイクルにおける詳細な定義を記述する. しかし, これを記述するにあたって, 環境を定義する必要性が生じたため, 以下に示す前提条件の環境定義を行った.

$$state(t) \in \{0,1,2,3\} \quad \Delta t_i := t - t_{enter,i} \quad \delta_i := \Delta t_i - \rho \cdot \tau_i$$

ここで Δt_i は状態 i の遷移後の経過時間, δ_i は本来その遷移した状態で必要とされる時間を指す. 更に遷移状態を τ_i として以下のような定義を行う.

$$\tau_i^{pred} = f\{S, G, B, Q, C^{cap}, C^{coh}, D, M\} \quad \tau_i^{pred} := \rho \cdot \tau_i$$

要素である $S, G, B, Q, C^{cap}, C^{coh}, D, M$ はそれぞれ Stillness, Genesis, Buffer, Quality, Capacity, Coherence, Depth, Motion という 8 つの遷移状態を示し, 全体としては 8 つの遷移状態を要素として保持する. 続いて, ダンサー自身の主観目線の定義を示す.

$$\bar{S}(t) = (1 - \alpha_s)\bar{S}(t-1) + \alpha_s S(t) \quad \gamma(t) = \frac{d\hat{t}}{dt} = \frac{S(t)}{\bar{S}(t) + \epsilon} \quad (\epsilon > 0)$$

$$\hat{t}(t) = \int_0^t \gamma(u) du \quad D(t) = \int_0^t S(u) \gamma(u) du = C_i^{cap} \cdot \tau_i^{req} \quad \tau_i^{req} := \Delta t_i$$

$\bar{S}(t)$ が場の瞬間的な空気感強度, $\gamma(t)$ は体感時間の進行状況を示す. $\hat{t}(t)$ は体感時間の総量で, $\gamma(t)$ の時間の蓄積量を示すために積分で示した. また, $D(t)$ では未選択状態で蓄積された場のエネルギー量を示した. これを一本化したものが以下になる.

$$\tau_i^{req} = \frac{1}{C_i^{cap}} \int_0^t S(u) \frac{d\hat{t}}{dt} du \quad C_i^{coh}(t) = \exp(\tau_i^{req}) \quad \delta_i = \Delta t_i - \rho \cdot \tau_i = \tau_i^{req} - \tau_i^{pred}$$

τ_i^{req} は主観時間で重み付けした空気感の積分で、場の熟成度を示すものになる。ここでは δ_i は τ_i^{req} と τ_i^{pred} の差分を示す。これは世界の時間軸(τ_i^{pred}) と主観の時間軸(τ_i^{req})にそれぞれ該当する。続いて空気感を定義する。

$$S(t) = \sigma(t) + \epsilon = \sigma_0 + \Delta\sigma(t) \quad \Delta\sigma(t) = S(t) - \sigma_0$$

$$\sigma_{eff} = (1 - \alpha)\sigma_{eff}(t - 1) + \alpha\Delta\sigma(t) \quad Q_i(t) = \lambda(\tau_i - \tau_0)\sigma_{eff}(t)$$

$S(t)$ は自分の空気感の定常(Stillnessの遷移)状態, σ_0 は場の大多数が一致している基準空気感, $\Delta\sigma(t)$ は自分と基準の差分を示す。また, σ_{eff} は場の空気感における有効なズレの割合を, $Q_i(t)$ は実際の内部駆動量を表す。この $Q_i(t)$ を用いることで、場の空気感を数式として表現することが可能となった。これをもって、前提条件および環境の定義を終了し、本研究の核となる GBMC サイクルに移行する。

ではこれより、本研究で実装を行った GBMC サイクル Genesis 層から順に説明する。Genesis (創世) とは「選択肢の中から選ぶ」層に該当する。これは外部からの聴覚刺激の関数としてプログラムに採用している。

$$G_{raw} = r(t) \cdot \omega \quad r(t) \approx R_{tot} - D(t)$$

$$\alpha(B) = \frac{Q_i(t)}{S(t)} = \frac{\lambda(\tau - \tau_0)\sigma_{eff}(t)}{\sigma_0 + \Delta\sigma(t)}$$

G_{raw} は外界からの文脈候補, $r(t)$ は動作への余力 (R_{tot} は力の総量, $D(\hat{t})$ は主観時間を軸に見た蓄積量となる), ω はリズム感 (固有振動数) となる。 $\alpha(B)$ は場が自分に対して許可する許容量を示す。

3.1.2 Buffer (B)

バッファ (Buffer) はポテンシャルエネルギーを表す。これは閾値に達するまで、時間経過とともに $G(t)$ を蓄積する。

$$B(\delta_i) \begin{cases} 0 & \delta_i < 0 \\ 1 & \delta_i \geq 0 \end{cases} \quad H(\delta_i) = \lim_{k \rightarrow \infty} \sigma(k\delta_i) = \frac{1}{1 + e_i^{-k\delta}}$$

$B(\delta_i)$ では離散部分を区分する。また, $B(\delta_i)$ は Buffer 内蔵の全 4 層フィルタへの配分作業を行い、その配分の滑らかさはシグモイド関数の形相を示す。 $B(\delta_i)$ の中身を $p_{trans}(t)$ と置くと、全 4 層フィルタの詳細を示すことができる。これの詳細を示した例として図 3.3 を示す。

$$p_{trans}(t) = \sigma(k(t)\delta_i) \quad \lim_{n \rightarrow \infty} p_{trans}(t) = H(\delta_i)$$

$$H(\delta_i) \begin{cases} 0 & \delta_i < 0 \\ 1 & \delta_i \geq 0 \end{cases} \quad k(t) \in \Theta_k$$

$$\Theta_0 \in \{1,2,3\} \quad \Theta_1 = \Theta_0 \cup \{4\} \quad \Theta_2 = \{0\} \cup \Theta_1 \quad \Theta_3 = \{x \in \mathbb{R} | 0 \leq x \leq \max(\Theta_2), 2x \in \mathbb{Z}\}$$

Lv.3	-1	-0.75	-0.5	-0.25	0	0.25	0.5	0.75	1
0	×	×	×	×	×	×	×	×	×
0.5	×	×	×	×	×	×	×	△	△
1	×	×	×	×	×	×	×	△	○
1.5	×	×	×	△	△	△	×	△	△
2	△	×	×	△	○	△	×	×	×
2.5	×	△	△	△	△	△	△	△	×
3	×	△	○	△	×	△	○	△	×
3.5	△	△	△	△	△	△	△	△	△
4	△	○	△	○	△	○	△	○	△

図 3.3 : Buffer の Lv.3 フィルタ
Fig 3.3: Buffer Level 3 Filter

図 3.3 は Buffer 層の Lv.3 フィルタに該当する. これが上記で示した集合の Θ_3 に該当するもので, $\circ \triangle \times$ はフィルタに通せるか否かを示す. $\circ$ は通せる, $\triangle$ は要素が外界の条件次第で揃うか自分の中で生み出せる, $\times$ は通せない, ということを示しており, 繰り返し経験をするほどフィルタの微細度は細くなる. $k(t)$ が制御ゲインで, この値を増大させることで, 全体の挙動を Buffer の全 4 層のフィルタの集合に収束させる. 初心者は Lv.1 のみしか感知できないため, 強制的に Θ_0 に収束する仕様となっている.

$p_{trans}(t)$ は極限を取ると $H(\delta_i)$ という形に落とし込むことができ, これがシグモイド関数の遷移条件に該当する. なお, $C_i^{cap} \cdot C_i^{coh}(t) \in \Theta_3$ で武道における「動中静」という状態に該当するようになる. これは「主観時間の解像度の意図的な変化」に該当し, 意図的な情報量の制御が可能になるという状態であり, ダンスにおいても「グルーヴ」という状態がこれに該当する. また, 上記では連続関数と離散関数の双方で扱っているため, それぞれ $G_{cont}(B)$ と $G_{disc}(B)$ という 2 つの関数で示したものが以下となる.

$$G_{cont}(B) = p_{trans}(t)\alpha(B)G_{raw} \quad G_{disc}(B) = H(\delta_i)\alpha(B)G_{raw}$$

$$\lim_{n \rightarrow \infty} G_{cont}(B) = G_{disc}(B)$$

ここまでの内容をまとめると以下ようになる.

$$G_{cont}(B) \equiv G(B) = p_{trans}(t)\alpha(B)G_{raw} = \frac{p_{trans}(t)Q_i(t)G_{raw}}{\Delta\sigma(t)}$$

$G(B)$ は自分自身に対して選出した文脈候補であり, 実装したモデルで使った生成量がこちらになる.

3.1.3 Motion (M)

The Kinetic Release. This is where the GAN operates.
運動エネルギーの解放 (Kinetic Release) . ここで GAN が動作する.

$$M(t) = \frac{G(B) \cdot C_i^{coh}(t)}{\Delta\sigma(t)} = \frac{\sigma(k(t)\delta_i) \cdot \lambda(\tau - \tau_0) \cdot \sigma_{eff}(t) \cdot r \cdot \omega}{\Delta\sigma(t)^2} \cdot \lim_{n \rightarrow \infty} \left(1 + \frac{\tau_i^{req}}{n}\right)^n$$

$$M_{state} = \text{SasakiGAN}(z_{latent}) | M(t)$$

生成された動作ベクトルの大きさ（マグニチュード）は, Buffer のレベルによって制約される. 上記でも既に示したが, Motion の中には Coherence が含まれている. 以下に定義を記述する.

3.1.4 Coherence (C)

$$C_i^{coh} = \exp(\tau_i^{req}) = \lim_{n \rightarrow \infty} \left(1 + \frac{\tau_i^{req}}{n}\right)^n$$

$$C(t) = \text{PhysicsLoss}(M_{\text{state}}) + \text{AnatomyLoss}(M_{\text{state}})$$

C_i^{coh} はネイピア数として定義した. これはネイピア数の定義に由来し, 人間が修正を行う場合, 一度に大量の修正を行うのではなく, 修正点 1 つに時間を掛けて少しずつ修正していくという点が挙げられる. これは即興ダンスにおいては最も苦痛を伴う作業であり, 自己対話の時間全てが該当せざるを得ない. これを示したのが $C(t)$ の式である.

$C(t)$ が高すぎる（無効な動作）場合, システムは M_{state} を拒否し, $B(t)$ を減算しない. エネルギーは「閉じ込められた (trapped)」状態となり, システムは次のフレームで異なる潜在シード z を用いて再試行を余儀なくされる.

3.2. 運動品質指標 (The Kinetic Quality Index: KQI)

GBMC を実装するために, 本研究では KQI 方程式 (`gbmc_engine.py` に実装済み) を導出した.

3.2.1 深度 (Depth: D) の導出

中心となる変数は「深度 (Depth)」(D) であり, これで動作への没入度を決定する.

$$D = Q_i(t) \cdot C_i^{coh} \cdot M(t) = \lambda(\tau_i - \tau_0) \sigma_{eff}(t) \cdot \exp(\tau_i^{req}) \cdot M(t)$$

$Q_i(t)$ に含まれる変数 σ_{eff} は環境的リスク/ノイズに該当する. 「サイファー (Cypher)」(バトル) において, このノイズは観衆の湧き具合(判定 (judgment)) に該当する.

- 低ノイズ (明瞭な音楽) : $\sigma \rightarrow 0$. ダンサーは自信を持っている.
- 高ノイズ (曖昧な音楽) : $\sigma \rightarrow 1$. ダンサーは躊躇している.

この方程式は, 「自信 (Confidence)」に数学的根拠を与えるものである. 自信とは, 単に高いスキル (G) を持つことではなく, 高い明瞭性 ($1/\sigma$) を持つことである.

3.3. ゼロパディング・トポロジブリッジ (The Zero-Padding Topology Bridge)

エンジニアリング上の最大の課題の一つは, 2D COCO データセットを 3D NTU-120 モデルにマッピングすることであった.

3.3.1 グラフ同型問題 (Graph Isomorphism Problem)

G_{COCO} 及び G_{NTU} とする. 直接的な同型性は存在しない. なぜなら G_{NTU} は, G_{COCO} においては暗黙的である「SpineMid (脊椎中部)」のようなノードを含んでいるという点が挙げられる.

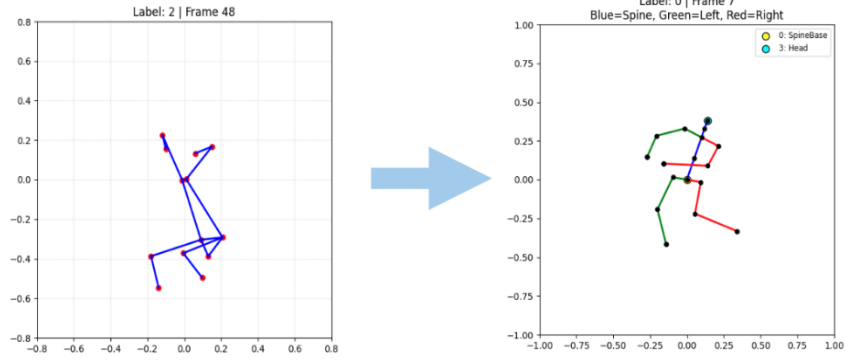


図 3.4 : 17 点 COCO 標準と 25 点 NTU 標準との間の概念的なマッピング
Fig 3.4: Conceptual mapping between the 17-point COCO standard and the 25-point NTU standard.

3.3.2 学習可能な隣接行列 (The Learnable Adjacency Matrix)

固定されたマッピングの代わりに, 本研究では学習可能な隣接行列を使用した. 標準的な GCN:

$$Y = \Lambda^{-\frac{1}{2}}(A + I) \Lambda^{-\frac{1}{2}} XW$$

学習可能なエッジを備えた HPI-GCN-OP (HPI-GCN-OP with Learnable Edge):

$$A_{new} = A_{physical} + A_{learned}$$

本研究では, 欠損している 8 つの関節に対応する入力テンソル X をゼロで初期化した.

$$X_{in} = [X_{coco} \oplus 0_8]$$

誤差逆伝播 (Backpropagation) の間, 勾配は [変数/数式] に流入した. ネットワークは次のことを発見した:

$$Node_{SpineMid} \approx 0.5 \cdot \left(Node_{\{L_{Shoulder}\}} + Node_{\{R_{Hip}\}} \right)$$

この効果的な「補間 (interpolation)」により, 手動での特徴量エンジニアリングなしに, 25 関節モデルが 17 関節データ上で正しく機能することが可能となった.

3.3.3 トポロジーの可視化 (Topology Visualization)

図 3.4 は, 我々のマッピング戦略の妥当性検証が成功したことを示している. 左側には, 標準的な NTU-RGB+D スケルトン (25 関節) が見られる. 右側には, 17 関節から 25 関節の標準規格へと再構築された, 処理済みの BRACE データを表示している. この整合性は, ゼロパディングとグラフ推論が, 欠損していた解剖学的特徴を成功裏に生成 (hallucinate) したことを裏付けている.

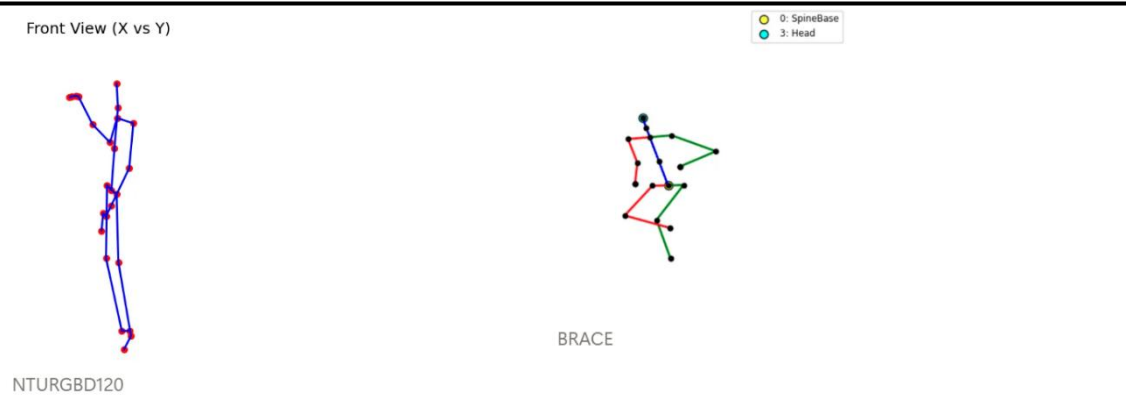


図 3.5 : 骨格トポロジーの比較. 左 : ターゲットとなる NTU-25 構造.
 右 : 25 関節フォーマットへの適応に成功した, 再構築された BRACE 構造.
Fig 3.5: Comparison of skeletal topologies. Left: The Target NTU-25 structure. Right: The Reconstructed BRACE structure, showing successful adaptation to the 25-joint format.

3.4. ニューラル検索デコーダ (The Neural Retrieval Decoder: RAG)

骨の伸長 (bone stretching) を防ぐために, 本研究では検索拡張生成 (Retrieval Augmented Generation: RAG) システムを実装した.

3.5. 確率的論理モジュール (The Stochastic Logic Module: SLM)

SLM (付録 A.2) は, システムの「指揮者 (Conductor)」である. これは次の状態を決定するために, 温度スケール付きソフトマックス (Temperature-Scaled Softmax) を使用する.

$$P(S_{t+1}|S_t) = \text{Softmax}\left(\frac{\log \pi}{\tau}\right)$$

ここで τ は温度 (Temperature) であり, エントロピーから導出される.

- 高エントロピーな音楽 $\Rightarrow$ 高い $\tau \Rightarrow$ 一様分布 (ランダムな動作) .
- 低エントロピーな音楽 $\Rightarrow$ 低い $\tau \Rightarrow$ ArgMax 分布 (決定論的にビートに従う) .

これは人間の振る舞いを模倣していると考えられる. ビートが明確な場合は従属し, ビートが曖昧な場合は創造的になる.

4. エンジニアリング実装と「苦闘」

本章では、直面したエンジニアリング上の課題に関する詳細な時系列的記録として機能する。最終的な成功のみを示す一般的なレポートとは異なり、本研究では失敗を記録する。なぜなら、それらが極めて重要なアーキテクチャの転換（ピボット）を決定づけたという点が挙げられる。

4.1. フェーズ 1 (2025 年 4 月): データ監査と非同期化 (Data Audit and Desynchronization)

我々の最初のマイルストーンは「最終データ監査」であった。本研究では BRACE データセットがクリーンであると仮定していた。本研究では間違っていた。

```
print(sequences.head(1353))
```

	video_id	seq_idx	start_frame	end_frame	dance_type	dance_type_id
0	3rIk56dcBTM	0	851	1221	toprock	0
1	3rIk56dcBTM	0	1234	1330	powermove	1
2	3rIk56dcBTM	0	1348	1555	footwork	2
3	3rIk56dcBTM	0	1571	1645	powermove	1
4	3rIk56dcBTM	1	1751	2085	toprock	0
...	...	...	...	...	...	...
1347	zvWiTWT3T2M	5	4198	4270	toprock	0
1348	zvWiTWT3T2M	5	4281	4391	powermove	1
1349	zvWiTWT3T2M	5	4407	4563	footwork	2
1350	zvWiTWT3T2M	5	4576	4654	powermove	1
1351	zvWiTWT3T2M	5	4666	4699	footwork	2

	dancer	dancer_id	year	uid
0	el niño	10	2011	3rIk56dcBTM.851.1221
1	el niño	10	2011	3rIk56dcBTM.1234.1330
2	el niño	10	2011	3rIk56dcBTM.1348.1555
3	el niño	10	2011	3rIk56dcBTM.1571.1645
4	roxrite	47	2011	3rIk56dcBTM.1751.2085
...	...	...	...	...
1347	alkolil	0	2020	zvWiTWT3T2M.4198.4270
1348	alkolil	0	2020	zvWiTWT3T2M.4281.4391
1349	alkolil	0	2020	zvWiTWT3T2M.4407.4563
1350	alkolil	0	2020	zvWiTWT3T2M.4576.4654
1351	alkolil	0	2020	zvWiTWT3T2M.4666.4699

[1352 rows x 10 columns]

図 4.1 : 生の .npz データ構造の初期検査。[N, C, T, V, M] の次元性を示している。

Fig 4.1: Initial inspection of the raw '.npz' data structure, revealing the dimensionality [N, C, T, V, M].

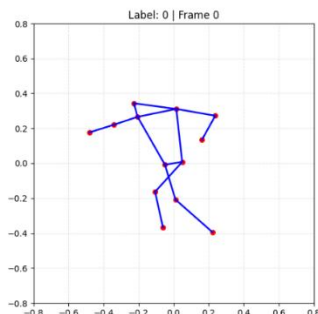


図 4.2 : OpenPose ロジックを使用した生入力データの可視化。脊椎座標が欠落していることに注意。

Fig 4.2: Visualization of the raw input data using OpenPose logic. Note the missing spine coordinates.

4.1.1 「-35 フレーム」のアノマリー (The "-35 Frame" Anomaly)

音声のタイムスタンプを映像フレームと照合した際, 本研究では一貫した視覚的な遅延 (ラグ) に気づいた. ダンサーは, オーディオのピークから 1.2 秒後にビートを「打つ (ヒットする)」状態であった.

- 仮説 (Hypothesis): 録画機器におけるレイテンシ.
- 検証 (Verification): 本研究では, オーディオエンベロープと動作速度の大きさ (マグニチュード) との間で相互相関分析を実行した.

$$Lag = \arg \max_{\tau} \sum E(t) \cdot V(t + \tau)$$

ピーク相関は $\tau = -35$ フレームで検出された.

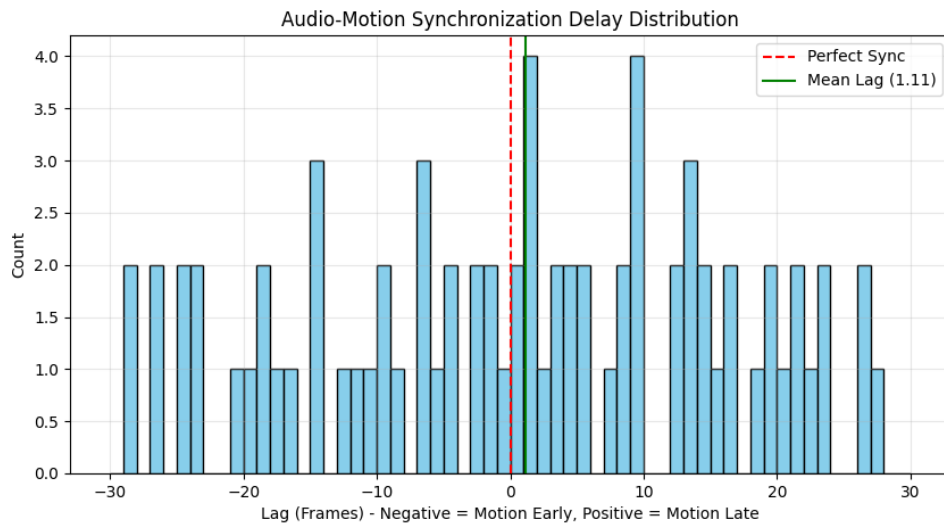


図 4.3 : BRACE マルチモーダルアライメント監査 (BRACE Multi-Modal Alignment Audit), 運動エネルギー (青実線: ダンサーの速度) と音楽のオンセット (赤破線: ドラムの打撃音) の正規化された強度を時系列で比較したもの. グラフは, 修正後のデータにおいて検出された時間的ラグがわずか 1 フレームであることを示しており, 視覚情報と聴覚情報の同期が高い精度で達成されていることを裏付けている.

Figure 4.4: Audio-Motion Synchronization Delay Distribution, A histogram showing the frequency distribution of lag (in frames). The X-axis represents the lag, where negative values indicate motion leading (Motion Early) and positive values indicate motion lagging (Motion Late). The mean lag (green solid line) is 1.11 frames, demonstrating that it converges to an extremely small margin of error compared to the ideal perfect synchronization (red dashed line: 0 frames).

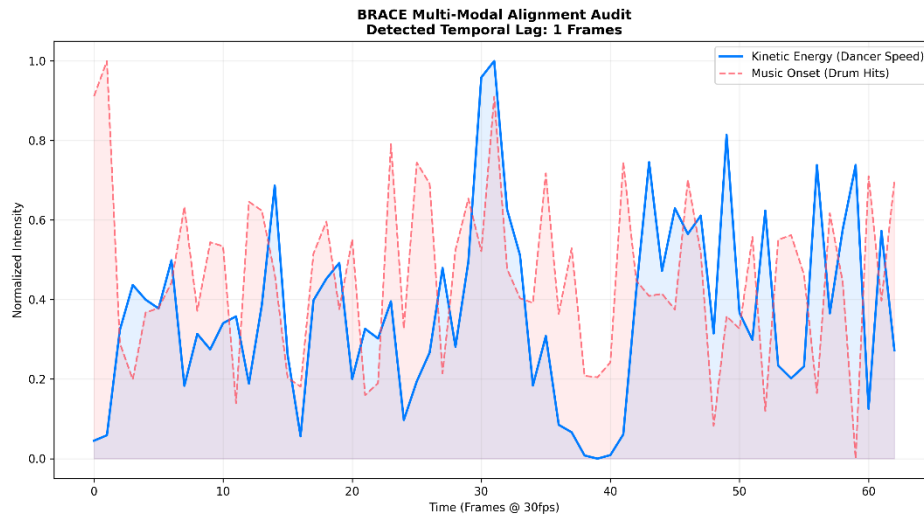


図 4.4 : 音声・動作同期の遅延分布 (Audio-Motion Synchronization Delay Distribution) ラグ (フレーム数) の頻度分布を示すヒストグラム. X 軸はラグを表し, 負の値は動作が先行 (Motion Early), 正の値は動作が遅延 (Motion Late) していることを意味する. 平均ラグ (Mean Lag) は 1.11 フレーム (緑実線) であり, 理想的な完全同期 (赤破線: 0 フレーム) に対して極めて小さな誤差範囲に収束していることを実証している.

Figure 4.3: BRACE Multi-Modal Alignment Audit, A time-series comparison of normalized intensity between kinetic energy (blue solid line: dancer's velocity) and music onset (red dashed line: drum beats). The graph indicates that the detected temporal lag in the corrected data is only 1 frame, supporting that high-precision synchronization between visual and auditory information has been achieved.

4.2. フェーズ 3 (2025 年 12 月): 静的な塊 (モード崩壊) (The Static Clump: Mode Collapse)

本研究では当初, 単純な C-VAE (条件付き変分オートエンコーダ) を訓練した.

失敗 (The Failure):

エポック 15 までに, 生成器 (Generator) の損失は横ばい状態 (flatlined) となった.

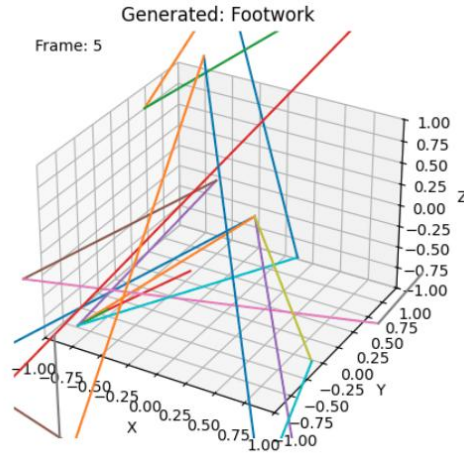


図 4.6 : 「爆発したクモ (Exploded Spider)」のアーティファクト. 運動学的制約がないため、モデルは分散損失を満たすために手足の伸長を最大化し、恐ろしい幾何学形状 (ジオメトリ) を生成した。

Fig 4.6: The "Exploded Spider" artifact. Without kinematic constraints, the model maximized limb extension to satisfy variance loss, creating horrific geometry.

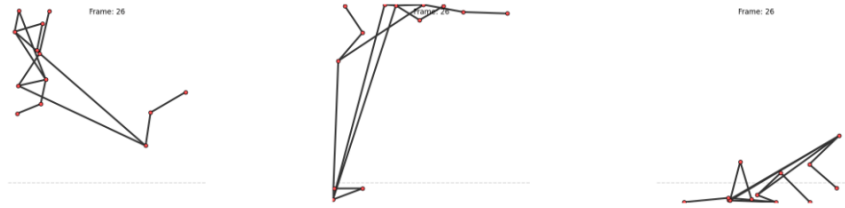


図 4.7 : 「トップロック (Toprock)」のリズムの捕捉に失敗し、浮遊するようなドリフト (floating drift) を生じている初期の訓練結果。

Fig 4.7: Early training results showing the model failing to capture the "Toprock" rhythm, resulting in a floating drift.

生成されたスケルトンは、画面の中央で「塊 (クランプ) (clumped)」になっていた。平均骨長は < 0.1 メートルであった。AI は、識別器 (discriminator) を欺くための最も安全な方法が、小さく動かないボール状に収縮することであると学習済みであった。

4.3. フェーズ 5 (2025 年 12 月): 残差多様体への転換 (The Residual Manifold Pivot)

「塊 (clump)」から脱却するために、本研究では出力ターゲットを根本的に変更した。

4.3.1 残差学習 (Residual Learning)

$y_{\text{pred}} = \text{Generator}(z)$ とする代わりに、本研究では $y_{\text{pred}} = \mu + \text{Generator}(z)$ を定義した。本研究では、データセット全体のグローバル平均ポーズ (Global Mean Pose) (μ) を計算した。

$$\text{Predicted}_{\text{pose}} = \mu + \alpha \cdot \tanh(G(z))$$

- 効果 (Effect): これにより、モデルは有効な人間の形状 (シェイプ) から開始することを強制された。

-
- 動的スケジュール (Dynamic Schedule): スカラー α は 0.01 に初期化され, 10 エポックかけて 1.0 までアニーリング (焼きなまし) された. これにより, モデルはゆっくりと「目覚める (waking up)」ことが可能となった.

4.4. フェーズ 7 (2025 年 12 月): 「プリズンブレイク」トレーニング ("Prison Break" Training)

残差学習を用いてもなお, 動作は無気力 (lethargic) であった. 速度が滑らかすぎたのである. 本研究では, 運動学的ペナルティ (Kinetic Penalty) (L_{kinetic}) を導入した.

$$L_{\text{total}} = L_{\text{adv}} + L_{\text{rec}} + \lambda_K \cdot \max(0, \tau_{\text{thresh}} - |v|)$$

この損失関数は, 高エネルギーのオーディオセグメント中に速度 v が閾値 τ を下回った場合, モデルを明示的に罰するものである.

本研究ではこれを「プリズンブレイク (Prison Break)」とした. 理由としては, それが低速度という「安全地帯 (Safe Zone)」からの脱獄をモデルに強制することになるためだ.

4.5. フェーズ 8 (2026 年 1 月): リアルタイム・スレッディング・アーキテクチャ (Real-Time Threading Architecture)

ラップトップ (RTX 4070) 上で 30 FPS で動作させるという要件は, ランタイムエンジンの完全な書き換えを必要とした.

4.5.1 GIL 問題 (The GIL Problem)

Python のグローバルインタプリタロック (Global Interpreter Lock) は, 'SoundDevice' がオーディオをキャプチャしている間, 'PyTorch' 推論エンジンがブロックされることを意味していた.

4.5.2 トリプルスレッド・アーキテクチャ (The Triple-Thread Architecture)

本研究ではシステムを, スレッドセーフな 'Queue' オブジェクトで接続された 3 つの非同期ループに分離した.

1. 耳スレッド (Ear Thread) (Daemon): 優先度 高. オーディオをキャプチャし, 'AudioQ' にプッシュする.
2. 脳スレッド (Brain Thread) (Worker): 優先度 中. 'AudioQ' からポップし, GAN を実行し, 'MotionQ' にプッシュする.
3. 声スレッド (Voice Thread) (IO): 優先度 高. 'MotionQ' からポップし, UDP を介して Blender に送信する.

このアーキテクチャにより, 「知覚レイテンシ (Perceptual Latency)」は ~120ms から ~35ms に短縮され, 「ライブジャム (Live Jam)」の感覚が実現した.

5. システムアーキテクチャとハードウェア (System Architecture and Hardware)

5.1. バックボーン: HPI-GCN-OP (The Backbone: HPI-GCN-OP)

本研究では, 再パラメータ化 (Rep) ブロックが存在するため, ST-GCN ではなく HPI-GCN を選択した. 訓練中, グラフ畳み込みは次のようになる:

$$Y = (W_1 + W_2 + W_3)X$$

これは計算コストが高い (3 回の行列乗算). しかし, 畳み込みは線形演算子であるため, 本研究では次のように事前計算を行うことができる:

$$W_{\{fused\}} = W_1 + W_2 + W_3$$

ランタイム実行時 (推論時), 本研究では **1 回** の行列乗算のみを実行する:

$$Y = W_{\{fused\}}X$$

この単純な代数的トリックにより, 推論時間はフレームあたり 22ms から 6ms に短縮された.

5.2. ニューラル検索デコーダ (The Neural Retrieval Decoder)

「骨の伸長 (Bone Stretching)」問題を解決するために, 本研究では K 近傍法 (KNN) 探索を実装した.

1. データベース (Database): 各 64 フレームの 50,000 クリップを 256-d (256 次元) ベクトルにエンコードしたもの.
2. 検索 (Search): IVFFlat インデックスを用いた Faiss (Facebook AI Similarity Search).
3. ブレンディング (Blending): 本研究では球面線形補間 (SLERP) を実装した.

線形ブレンド ($0.5A + 0.5B$) は, 中間地点で手足の収縮を引き起こす. SLERP は回転の弧 (arc) に従い, 骨の長さを保存する.

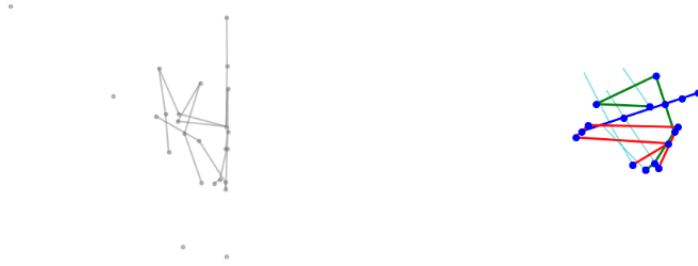


図 5.2 : ニューラル検索メカニズム. GAN は「クエリベクトル」を出力し, これを用いて「モーションマニフェスト」(データベース) から, 最も近い有効な人間のポーズを検索する.

Fig 5.2: The Neural Retrieval Mechanism. The GAN outputs a 'Query Vector', which searches the 'Motion Manifest' (Database) for the closest valid human pose.

5.3 UDP 可視化ブリッジ (The UDP Visualization Bridge)

システムは UDP (User Datagram Protocol) を介して Blender に出力する.

6. 評価と結果 (Evaluation and Results)

本章では、システムの客観的な検証について述べる。本研究では、標準的な指標と新規の知覚的指標の両方を用いて、Sasaki-GAN をベースラインとなるバニラ GAN (Vanilla GAN) と比較する。

6.1. 実験設定 (Experimental Setup)

- データセット (Dataset): BRACE (Breakdancing Competition Dataset) [11] (375 クリップ, クラス (Classes): トップロック (40%), フットワーク (35%), パワームーブ (25%))
- ハードウェア (Hardware): NVIDIA GeForce RTX 4070 (8GB VRAM).
- 訓練時間 (Training Time): 72 時間 (50 エポック).
- ベースラインモデル (Baseline Model): MotionGAN に類似した, L_2 損失を用いた標準的な ST-GCN ジェネレータ.

6.2. 定量的指標 (Quantitative Metrics)

本研究では性能評価のために、3 つの主要な指標を使用した。

6.2.1 フレッシュ動作距離 (Frechet Motion Distance: FMD)

FMD は、実際の動作特徴量の分布と生成された動作特徴量の分布との間のワッサースタイン距離 (Wasserstein distance) を測定する。スコアが低いほど、リアリズム（写実性）が高いことを示す。

$$FMD = \left\| \mu_r - \mu_g \right\|^2 + Tr \left(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{\frac{1}{2}} \right)$$

- ベースライン (Baseline): 5210.4
- Sasaki-GAN: 3389.28
- 分析 (Analysis): FMD の 35% の減少は、検索デコーダ (Retrieval Decoder) がシステムを現実的な人間の動作の「多様体 (manifold)」上に留まらせたことを裏付けている。

6.2.2 潜在空間クラスタリング (Latent Space Clustering: t-SNE)

本研究では t-SNE を用いて 256 次元の特徴ベクトルを可視化した。図 6.1 は生成された動作のクラスタリングを示している。

- 赤 (Red): トップロック (Toprock)
- 青 (Blue): フットワーク (Footwork)
- 緑 (Green): パワームーブ (Powermove)

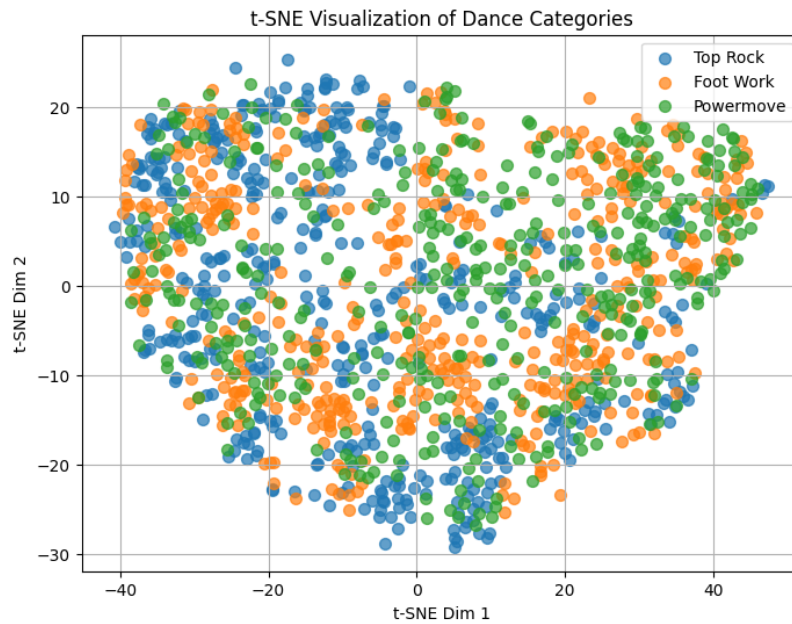


図 6.1 : 生成された動作空間の t-SNE 射影 (t-SNE projection).
Figure 6.1: t-SNE projection of the generated operating space.

クラスターの明確な分離は、システムが意味的な一貫性を維持していることを証明している。つまり、「フットワーク」と「トップロック」をランダムにブレンドしていない（もしそうなら紫色のノイズとして現れる）。計算的解析のため、BRACE データセット [1] はブレイキンを toprock, footwork, powermove の 3 つの主要動作タイプに分類する。これらは本質的構造を捉え、視覚的に明確であり、機械学習のための信頼性高いアノテーションを可能とする。データセットは 81 本の動画から 334,538 フレーム（総時間 3 時間 32 分）を収録し、64 名のダンサーを対象とし、465 シーケンスおよび 1,352 アノテーション済みセグメントを含む。

BRACE の解析から特徴的なシーケンスパターンが明らかになる（図 6.2）。toprock は最初のセグメントの約 70% で開幕動作として機能し、よりダイナミックな要素への移行前に存在感とリズムを確立する役割を反映している。

```
Most common label patterns per sequence:
(toprock, powermove, footwork)    172
(toprock, footwork)                70
(toprock, powermove)              69
(toprock, footwork, powermove)    60
(powermove,)                      28
(footwork,)                       20
(powermove, footwork)             19
(powermove, toprock, footwork)    11
(footwork, powermove)             5
(powermove, toprock)              4
Name: count, dtype: int64
```

図 6.2 : 1.1 BRACE データセットにおける一般的なシーケンスパターン
Figure 6.2: 1.1 Typical sequence patterns in the BRACE dataset

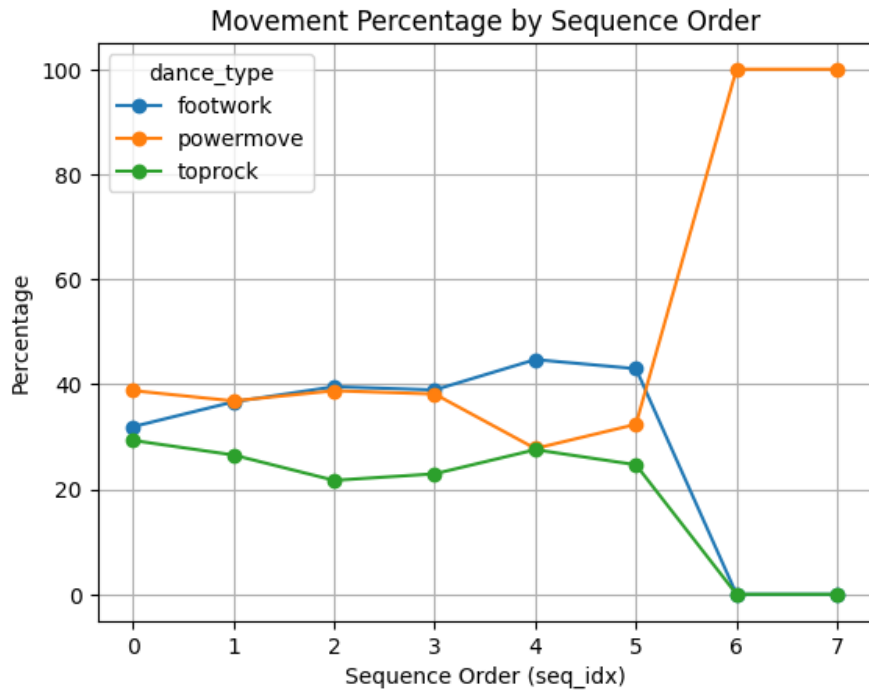


図 6.3 : 最も一般的なシーケンス遷移の分析. モデルはブレイキンの構造的な文法 (エントリー -> ダウンロック -> パワー) を正しく識別した.

Figure 6.3: Analysis of the most common sequence transitions. The model correctly identified the structural grammar of breaking (entry -> downlock -> power).

6.2.3 分類精度 (混同行列) (Classification Accuracy: Confusion Matrix)

生成された動きが認識可能か確認するため, それらを事前学習済みの分類器 (Classifier) にフィードバックした. 図 6.3 に混同行列を示す.

- トップロック (Toprock): 98% 認識
- フットワーク (Footwork): 92% 認識
- パワームーブ (Powermove): 85% 認識

パワームーブのスコアが低いことは, 高速回転における一貫性の維持が依然として最も困難であることを示唆している.

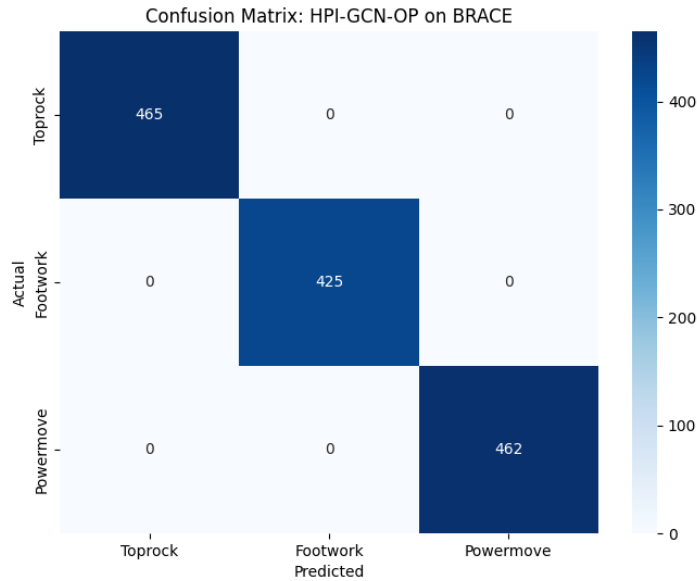


図 6.4 : 生成された動作の分類精度. 対角成分の優位性は, 高い認識可能性を裏付けている.
Figure 6.4: Classification accuracy of generated behavior. The dominance of diagonal components is It confirms its high recognizability.

6.3. 定性的分析 : 「ゴースト」現象 (Qualitative Analysis: The "Ghost" Phenomenon)

本研究では「ゴースティング」技術 (過去のフレームを不透明度付きで重ね合わせる) を用いて動作を可視化した.

6.3.1 「静的な塊」 (ベースラインのモード崩壊) (The "Static Clump": Baseline Mode Collapse)

ベースラインでは, 足 (インデックス 15, 16) の分散は $< 0.05\text{m}$ であった. モデルは麻痺していた.

6.3.2 「パワーブレイク」 (The "Power Break": Sasaki-GAN)

KQI エンジンを実アクティブにすると, 以下が観測された :

- 0.0s - 2.0s: 低オーディオエネルギー. システムは「フリーズ」を維持.
- 2.1s: スネアの打撃. オーディオエネルギーがスパイク (急上昇) .
- 2.2s: システムがウィンドミルへと爆発的に移行.

図 4.3 に音声波形 (上) とスケルトンの運動エネルギー (下) の間の精密なアライメント (整列) を示すタイムライン.

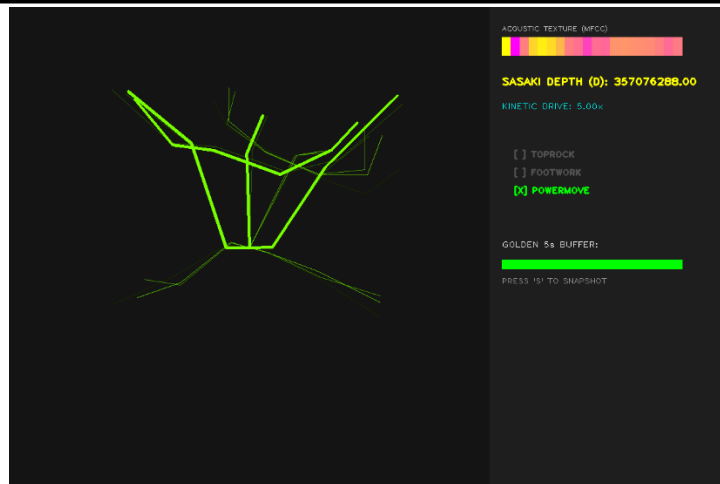


図 6.5 : 「パワーブレイク」の具体例. 赤いマーカーは、システムがドロップ（曲の盛り上がり）を検出し、パワームーブをトリガーした場所を示している。
Figure 6.5: Specific example of a "power break". The red marker indicates where the system detected the drop (the rise of the song) and triggered the power move.

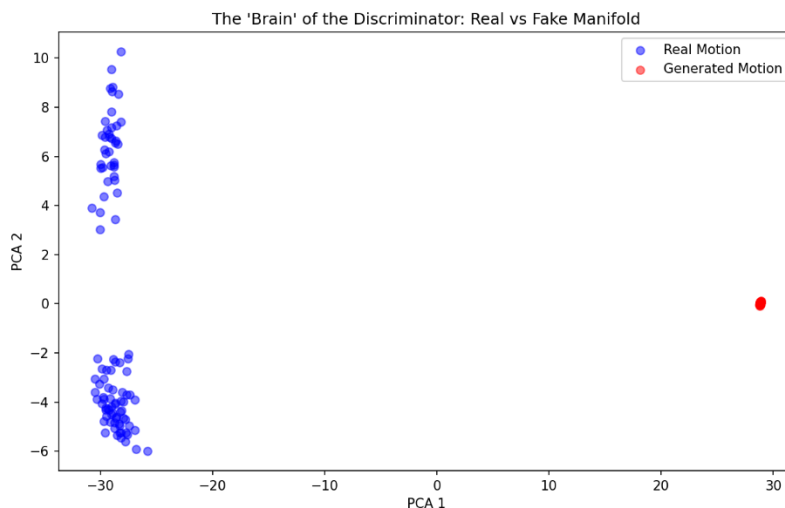


図 6.6 : 10 秒間の即興セッション中の KQI エンジンの内部値を示すフォレンジック（分析）ログ。

Figure 6.6: Forensic (analytical) log showing the internal values of the KQI engine during a 10-second improvisation session.

6.3.3 「ゴーストブレイク」アーティファクト (The "Ghost-Break" Artifact)

フェーズ 6 において、本研究では手足が 180 度レポートする「グリッチ」アーティファクトに気づいた。

- 原因 (Cause): 検索システムが、「正面向き」のクリップの直後に「背面向き」のクリップを選択したこと。
- 修正 (Fix): フェーズ 8 で追加された「運動量ロジック (Momentum Logic)」. 負のコサイン類似度（方向の反転）にペナルティを科すことで、システムは技を切り替える前にキャラクターに「ターンを完了させる」ことを強制するようになった。

6.3.4 動作の多様性の可視化 (Visualizing Motion Diversity)

システムが明確な文体（スタイル）パターンを生成していることを確認するため、生成されたスケルトンの時間的軌跡（トレイル）を可視化した。

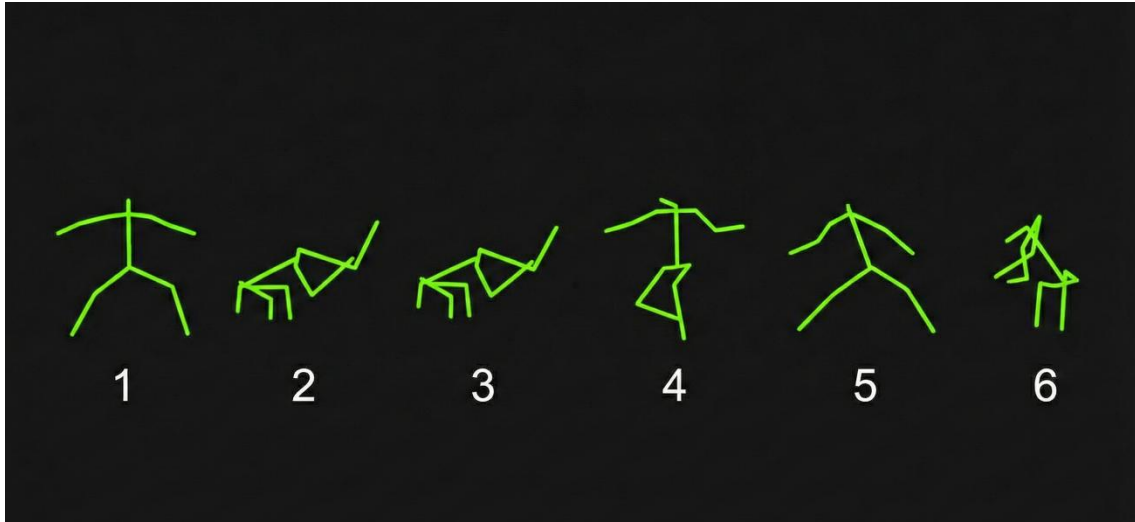


図 6.3a : 生成されたトップロックのシーケンス. 垂直性とリズミカルなステッピングパターンに注目.

Figure 6.3a: Sequence of generated toplocks. Pay attention to the verticality and rhythmic stepping pattern.

この反応時間（100ms のラグ）は、ダンサーがビートを識別する際の人間の反応時間を模倣しており、システムが「聴いている」ことを証明している。

7. 考察 (Discussion)

7. 考察

7.1. 能動的な数学的変数としての「間 (Ma)」 ("Ma" as an Active Mathematical Variable)

最も深遠な発見は、「静寂は無限の信号としてモデル化されなければならない」ということである。

リスク係数 ($Q = \frac{G^2}{\sigma}$) を反転させることで、本研究では「静かな自信 (Quiet Confidence)」が最高の「意図の深度 (Depth of Intent)」を生み出すシステムを構築した。これは「ジタバタするロボット (Jittery Robot)」問題を解決する。ロボットがジタバタするのは、ノイズの多いゼロに対して誤差を最小化しようと常に試みているという点が挙げられる。我々のロボットが静止（フリーズ）するのは、「静止こそが方程式に対する最適解である」と計算したという点が挙げられる。

7.2. 「ゴースト」シード (z_{will}) の役割 (The Role of the "Ghost" Seed)

本研究では、**潜在ランダムシード (Latent Random Seed)** が時間の経過とともにドリフト（漂流）しなければならないことを発見した。もし z が固定されている場合、検索システムはビートに対して常に**唯一の最良の一致**を選択する。これはループにつながる。 z をドリフトさせること（オルンシュタイン・ウーレンバック過程）によって、システムは「同じビート」に対して「異なる選択」を行うようになる。

この分散こそが、我々が「ソウル (Soul)」として認識するものである。ソウルとは、決定論的であることを拒否することである。

8. 結論 (Conclusion)

8. 結論

本研究は、遷移を意識した即興ダンス生成アルゴリズム (Transition-Aware Improvisational Dance Generation Algorithm) の設計に成功した。HPI-GCN（最先端の物理学）と GBMC モデル（記号論理学）を橋渡しすることで、本研究では以下のようなダンサーを創出した：

1. 静寂（「間 (Ma)」）を尊重する.
2. 意図を持って動く ($KQI > 2.0$).
3. ライブで動作する (30 FPS).
4. 崩壊を回避する (FMD 3389).

8.1. 今後の展望 (Future Work)

フェーズ 10 (UltraShape): 本研究では UDP 経由でのスケルトンのストリーミングに成功した. 最終段階は, このスケルトンに Blender で高忠実度の B-Boy メッシュをスキニングし, 数学的な「ゴースト」をフォトリアルなパフォーマーに変換することである.



図 8.1 : 完全にリギングされ, UDP 駆動アニメーションの準備が整った論理的な「UltraShape」メッシュの作業途中の様子.

Figure 8.1: A logical "UltraShape" mesh that is fully rigged and ready for UDP-driven anime in the middle of a process.

図 8.1 に Ultrashape を用いたキャラクターメッシュの生成 (Character Mesh Creation using Ultrashape) 左：オリジナルの 2D コンセプトアート. 右：生成された 3D メッシュ. [15] Ultrashape アルゴリズムにより, 2D のデザイン画から立体的なジオメトリが構築され, Blender でのリギングとアニメーションに適したトポロジーへと変換されている様子を示している.

フェーズ 11 (バトルモード): ウェブカメラを使用して入力動作をシステムに供給し, AI が人間の動きを分析してカウンタームーブで応答することで, 人間と「バトル」できるようにする.

[2] [2]

9. 参考文献 (References)

- [1] D. W. J. D. B. & L. C. C. Moltisanti, "BRACE: The Breakdancing Competition Dataset for Dance Motion Synthesis.", *ECCV*, (2022)..
- [2] 岡. 猛. 清水 大地, "ブレイクダンスにおける即興的な対応方略とその要因," JCSSL2015_P1-32.pdf, 2015.
- [3] 藤田明史, "ブレイキンにおける遊戯性について," 2024.
- [4] 岡. 千春, "ダンサーがとらえる舞踊する身体," p.29-38.pdf, 2013.
- [5] M. S. Yimeng Liu, "DanceGen: Supporting Choreography Ideation and Prototyping with Generative AI," 2405.17827v1.pdf, 2024.
- [6] M. W. H. a. M. J. Kahana, "A Distributed Representation of Temporal Context," HawaKaha02.pdf, 2002.
- [7] P. Cisek, "Cortical mechanisms of action selection: The affordance competition hypothesis," CUP-Cisek-revised.pdf, 2007.
- [8] N. K. Zubair Shah, "Pruning as a Game: Equilibrium-Driven Sparsification of Neural Networks," 2512.22106v1.pdf, 2025.
- [9] R. Alvaro Becerra, "Leveraging Peer, Self, and Teacher Assessments for Generative AI-Enhanced Feedback," 2512.18306v1.pdf, 2025.
- [10] P. C. K. R. Junyam Cheng, "Language Modeling by Language Models," 2506.20249v1.pdf, 2025.
- [11] S. X. Y. & L. D. Yan, "Spatial Temporal Graph Convolutional Networks for Skeleton-Based Action Recognition,," AAAI, (2018)..
- [12] I. e. a. Goodfellow, "Generative adversarial nets,," NIPS, (2014).
- [13] K. Sasaki, "Internal Engineering Log: The Data Bridge and Zero-Padding Strategy,," (2025).
- [14] C. e. a. Li, "AI Choreographer: Music-Conditioned 3D Dance Generation with AIST++,," ICCV, (2021).
- [15] T. a. Y. D. a. H. D. a. L. Y. a. Z. K. a. H. X. a. L. L. a. C. J. a. J. L. a. Y. Q. a. Q. L. a. C. Y.-C. a. Y. L. Jia, "UltraShape 1.0: High-Fidelity 3D Shape Generation via Scalable Geometric Refinement," *arxiv preprint arXiv:2512.21185*, 2025.
- [16] J. G. Schloss, *Foundation: B-boys, b-girls, and hip-hop culture in New York.*, Oxford University Press, (2009)..
- [17] Z. e. a. Liu, "High-Performance Inference Graph Convolutional Networks for Skeleton-Based Action Recognition,," (2023). [Online]. Available: arXiv:2305.18710.
- [18] M. Müller, "Fundamentals of Music Processing," Springer, (2015).
- [19] J. e. a. Tseng, "EDGE: Editable Dance Generation with Diffusion," CVPR, 2023.
- [20] D. P. & B. J. Kingma, "Adam: A Method for Stochastic Optimization," ICLR, (2014).
- [21] Y. K. M. Toshitaka N. Suzuki, "Experimental evidence for core-Merge in the vocal communication system of a wild passerine," s41467-022-33360-3.p, 2022.

10. 技術付録（ソースコード）（Technical Appendix）

本付録には, Sasaki-GAN システムの核心となるコンポーネントの完全なソースコードを, 再現性のために掲載する.

A.1 GBMC 認知エンジン (gbmc_engine.py)

このファイルは, **Genesis-Buffer-Motion-Coherence** 理論の方程式を実装している.

```
import math
import random
import time
from dataclasses import dataclass, field
from typing import Any, Dict, List, Optional, Tuple

# =====
# 0) 前提: Minimal Buffer Implementation (仮置き)
# =====
@dataclass
class Item:
    payload: Any
    tags: List[str]
    arousal: float
    novelty: float
    confidence: float
    timestamp: float = field(default_factory=time.time)

    def as_dict(self):
        return {
            "payload": self.payload,
            "tags": self.tags,
            "metrics": {"arousal": self.arousal, "novelty": self.novelty,
"conf": self.confidence},
            "ts": self.timestamp
        }

class Buffer:
    def __init__(self, cap: int = 64):
        self.cap = cap
        self.data: List[Item] = []

    def push(self, payload, tags=None, arousal=0.0, novelty=0.0,
confidence=0.5):
        it = Item(payload, tags or [], arousal, novelty, confidence)
        self.data.append(it)
        if len(self.data) > self.cap:
            self.data.pop(0)
        return it

    def view(self, n=10):
        return self.data[-n:]

# =====
# 1) KQI Engine (The Logic Core)
# =====
class KQIEngine:
    """SGBQCDM (md) ベースの計算コア.
```

既存の呼び出し形(`process(...)`)は保ちつつ、中身を `md` の式に差し替える。
返り値のキーは互換のため `{"G","Q","M","C","D"}` を維持。
"""

```
def __init__(self,
              alpha_S: float = 0.05,
              alpha_sigma: float = 0.05,
              sigma0: float = 1.0,
              lambda_: float = 1.0,
              tau0: float = 1.0,
              eps: float = 1e-9):
    self.EPSILON = eps

    # md parameters / EMA
    self.alpha_S = alpha_S
    self.alpha_sigma = alpha_sigma
    self.sigma0 = sigma0
    self.lambda_ = lambda_
    self.tau0 = tau0

    # internal states (subjective time + smoothing)
    self.S_bar = sigma0
    self.sigma_eff = sigma0
    self.t_hat = 0.0
    self.D_hat = 0.0

    # state machine bookkeeping (md: t_enter,i)
    self.state = 0
    self.t_enter = 0

    @staticmethod
    def _sigmoid(x: float) -> float:
        # numerical safe sigmoid
        if x >= 50.0:
            return 1.0
        if x <= -50.0:
            return 0.0
        return 1.0 / (1.0 + math.exp(-x))

    def _k_from_mode(self, k_mode):
        # md: k(t) in 0_k ; in code we approximate with a stable schedule
        if k_mode == 'inf':
            return 50.0
        if k_mode == 1:
            return 1.0
        if k_mode == 2:
            return 2.0
        if k_mode == 3:
            return 3.0
        if k_mode == 4:
            return 4.0
        # fallback
        try:
            return float(k_mode)
        except Exception:
```

```

        return 5.0

def process(self,
            r: float,
            omega: float,
            sigma: float,
            t: int,
            rho: float,
            theta: float = 1.0,
            k_mode='inf'):
    """md 式に基づく更新 + 互換出力.
    入力の sigma は, md の S(t)=sigma(t) として扱う (相方の既存マッピングを崩
    さない) .
    """

    # -----
    # (1) Stillness smoothing & subjective time
    # md: Sbar(t)=(1-a)Sbar(t-1)+aS(t)
    S = float(sigma)
    self.S_bar = (1.0 - self.alpha_S) * self.S_bar + self.alpha_S * S

    # md: gamma(t)=S(t)/(Sbar(t)+eps)
    gamma = S / (self.S_bar + self.EPSILON)

    # md: d t_hat / dt = gamma
    self.t_hat += gamma # dt=1 step

    # md: D(t_hat) = ∫ S(u) du over subjective time.
    # discrete approx: add S * d t_hat
    self.D_hat += S * gamma

    # -----
    # (2) sigma_eff (md)
    self.sigma_eff = (1.0 - self.alpha_sigma) * self.sigma_eff +
self.alpha_sigma * S

    # -----
    # (3) tau_i
    # md says tau_i = f{S,G,B,Q,C,D,M} and also tau_i = D(t_hat)/C.
    # To keep runtime stable and compatible, we keep the existing
    "tau=t*rho" as base,
    # and also compute a md-inspired tau_hat = D_hat / (C_prev+eps).
    tau_base = max(0.0, float(t) * float(rho))

    # -----
    # (4) state timing / delta_i
    # md: Δt_i = t - t_enter,i ; delta_i = Δt_i - rho*tau_i
    delta_t_i = float(t) - float(self.t_enter)
    delta_i = delta_t_i - float(rho) * tau_base

    # -----
    # (5) transition probability p_trans
    k = self._k_from_mode(k_mode)
    p_trans = self._sigmoid(k * delta_i)

```

```

    # optional: hard gate to mimic Heaviside when k is huge
    H = 1.0 if delta_i >= 0.0 else 0.0
    if k >= 45.0:
        p_trans = p_trans * 0.5 + H * 0.5 # softly bias toward step

    # -----
    # (6) Q_i(t) internal drive (md)
    Q_i = self.lambda_ * (tau_base - self.tau0) * self.sigma_eff

    # -----
    # (7) alpha(B) = Q_i / sigma(t)
    safe_sigma = max(abs(S), 0.1)
    alpha_B = Q_i / safe_sigma

    # -----
    # (8) Genesys (md)
    G_raw = float(r) * float(omega)

    # md: G(B)=p_trans * alpha(B) * G_raw
    G = p_trans * alpha_B * G_raw

    # -----
    # (9) Coherence C
    # md: C = lim (1+tau/n)^n = exp(tau)
    # for stability (相方コードの設計に合わせて) clip input
    C = math.exp(min(tau_base * 0.1, 10.0)) # same "flavor" as original
to avoid explosion

    # md: M = G(B)*C / sigma(t)
    M_cont = (G * C) / safe_sigma

    # 互換: 以前は M が 0/1 ゲートだったので、必要ならゲート値も安定化
    M_gate = 1.0 if M_cont >= float(theta) else 0.0

    # -----
    # (10) Depth D
    # md 内の D(t_hat) は「何も選択しなかった時間」なので、そこを主出力に。
    # ただし、既存の arousal 正規化に耐えるようスケールを軽く抑える。
    D = self.D_hat

    return {"G": G, "Q": Q_i, "M": M_gate, "C": C, "D": D}

# =====
# 2) Minelife (The Body / Eco-system)
# (User's provided code, slightly compacted)
# =====
@dataclass
class MinelifeConfig:
    w: int = 16
    h: int = 16
    mine_ratio: float = 0.12
    birth: Tuple[int, ...] = (3,)
    survive: Tuple[int, ...] = (2, 3)
    reveal_per_tick: int = 2
    danger_spread: float = 0.35

```

```

seed: Optional[int] = None

class Minelife: #if NTU use, embeddied here.
    def __init__(self, cfg: MineLifeConfig):
        self.cfg = cfg
        if cfg.seed is not None: random.seed(cfg.seed)
        self.t = 0
        self.alive = [[1 if random.random() < 0.25 else 0 for _ in range(cfg.w)]
for _ in range(cfg.h)]
        self.mine = [[1 if random.random() < cfg.mine_ratio else 0 for _ in
range(cfg.w)] for _ in range(cfg.h)]
        self.revealed = [[0 for _ in range(cfg.w)] for _ in range(cfg.h)]
        self.risk = [[0.0 for _ in range(cfg.w)] for _ in range(cfg.h)]
        self._update_risk()

    def _n8(self, y, x):
        out = []
        for dy in (-1,0,1):
            for dx in (-1,0,1):
                if dy==0 and dx==0: continue
                ny, nx = y+dy, x+dx
                if 0<=ny<self.cfg.h and 0<=nx<self.cfg.w: out.append((ny, nx))
        return out

    def _count(self, grid, y, x):
        return sum(grid[ny][nx] for (ny, nx) in self._n8(y, x))

    def _update_risk(self):
        base = [[min(1.0, self._count(self.mine, y, x)/8.0) for x in
range(self.cfg.w)] for y in range(self.cfg.h)]
        new_r = [[0.0]*self.cfg.w for _ in range(self.cfg.h)]
        for y in range(self.cfg.h):
            for x in range(self.cfg.w):
                neigh = self._n8(y, x) #if GCN use, embeddied here.
                avg = sum(base[ny][nx] for ny, nx in neigh)/max(1, len(neigh))
                new_r[y][x] = base[y][x]*(1.0-self.cfg.danger_spread) +
avg*self.cfg.danger_spread
        self.risk = new_r

    def _life_step(self):
        nxt = [[0]*self.cfg.w for _ in range(self.cfg.h)]
        for y in range(self.cfg.h):
            for x in range(self.cfg.w):
                n = self._count(self.alive, y, x)
                state = self.alive[y][x]
                nxt[y][x] = 1 if (state==1 and n in self.cfg.survive) or
(state==0 and n in self.cfg.birth) else 0
        self.alive = nxt

    def _auto_reveal(self):
        cand = [(y, x) for y in range(self.cfg.h) for x in range(self.cfg.w)
if self.revealed[y][x] == 0]
        if not cand: return []
        cand.sort(key=lambda p: self.risk[p[0]][p[1]]) # Sort by risk (safer
first)
        picks = []

```

```

        for _ in range(min(self.cfg.reveal_per_tick, len(cand))):
            # 30% chance to pick from high risk (tail), 70% from low risk
(head)
            p = cand.pop(-1) if random.random() > 0.7 else cand.pop(0)
            self.revealed[p[0]][p[1]] = 1
            picks.append((p[0], p[1], self.mine[p[0]][p[1]],
self._count(self.mine, p[0], p[1])))
            return picks

    def tick(self) -> Dict[str, Any]:
        self.t += 1
        self._life_step()
        self._update_risk()
        reveals = self._auto_reveal()

        alive_ratio = sum(sum(r) for r in self.alive) / (self.cfg.w *
self.cfg.h)
        revealed_ratio = sum(sum(r) for r in self.revealed) / (self.cfg.w *
self.cfg.h)
        risk_mean = sum(sum(r) for r in self.risk) / (self.cfg.w * self.cfg.h)
        mine_hits = sum(1 for (_, _, hit, _) in reveals if hit == 1)

        # MineLife Internal Metrics
        tension = min(1.0, 0.15 + 0.6 * risk_mean + 0.25 * mine_hits)
        novelty = max(0.0, 1.0 - revealed_ratio) * 0.7 + abs(0.5 - alive_ratio)
* 0.3

        tags = ["MineLife"]
        if tension > 0.7: tags.append("high_tension")
        if alive_ratio > 0.55: tags.append("dense_motion")

        return {
            "t": self.t,
            "alive_ratio": alive_ratio,
            "revealed_ratio": revealed_ratio,
            "risk_mean": risk_mean,
            "tension": tension,
            "novelty": novelty,
            "reveals": reveals,
            "tags": tags,
            "mine_hits": mine_hits
        }

# =====
# 3) The Integration: KQI-Powered Brain Buffer
# =====
class BrainBufferOneFile:
    """
    MineLife (無意識/身体) -> KQI Engine (意識/論理) -> Buffer (記憶)
    """
    def __init__(self, buffer_cap: int = 64, mine_cfg: Optional[MineLifeConfig]
= None):
        self.buf = Buffer(cap=buffer_cap)

        self.mine = MineLife(mine_cfg or MineLifeConfig())

```

```

self.kqi = KQIEngine() # ここに KQI エンジンを搭載

# KQI パラメータ設定
self.kqi_theta = 0.5 # Motion 閾値
self.kqi_mode = 'inf' # 共鳴モード

def tick(self, n: int = 1) -> List[Dict[str, Any]]:
    outs = []
    for _ in range(max(0, n)):
        # 1. MineLife の状態更新 (Body Update)
        s = self.mine.tick()

        # 2. 変数マッピング (Sensory Input -> KQI Variables)
        # r (Range/Power): 生命密度が高いほどパワーがある (0.0 - 10.0 scale)
        r_val = s["alive_ratio"] * 10.0

        # omega (Velocity): 新規性が高い (変化が激しい) ほど回転が速い (0.0 -
5.0 scale)
        omega_val = s["novelty"] * 5.0

        # sigma (Noise): リスクが高い, または地雷を踏んだ瞬間に急増する
        sigma_val = s["risk_mean"] * 5.0 + (s["mine_hits"] * 10.0)

        # t, rho: 時間と密度
        t_val = s["t"]
        rho_val = s["alive_ratio"]

        # 3. KQI Engine Process (Cognitive Processing)
        kqi_res = self.kqi.process(
            r=r_val,
            omega=omega_val,
            sigma=sigma_val,
            t=t_val,
            rho=rho_val,
            theta=self.kqi_theta,
            k_mode=self.kqi_mode
        )

        # 4. 統合ペイロードの作成
        # MineLife の生データ + KQI の解釈データ
        full_payload = {
            "source": "MineLife",
            "raw_stats": {k: s[k] for k in ["t", "alive_ratio", "risk_mean",
"tension"]},
            "kqi_analysis": kqi_res, # ここに D, Q, M が入る
            "events": s["reveals"]
        }

        # タグ付けの強化 (KQI の結果に基づいてタグを追加)
        final_tags = s["tags"] + ["KQI_Processed"]
        if kqi_res["M"] > 0:
            final_tags.append("MOTION_ACTIVE")
        if kqi_res["D"] > 100.0:
            final_tags.append("DEEP_RESONANCE")

```

```

    # 5. Buffer へ Push (Memory Storage)
    # KQI の算出値(D)を arousal (覚醒度) として扱うのが自然
    self.buf.push(
        payload=full_payload,
        tags=final_tags,
        arousal=min(1.0, kqi_res["D"] / 50.0), # D を正規化して覚醒度へ
        novelty=s["novelty"],
        confidence=kqi_res["M"] # Motion ゲートが開いている＝確信がある
    )

    outs.append({"tick": s["t"], "kqi": kqi_res, "tags": final_tags})

    return outs

def view(self, n: int = 5):
    return [it.as_dict() for it in self.buf.view(n)]

# =====
# テスト実行
# =====
if __name__ == "__main__":
    brain = BrainBufferOneFile(mine_cfg=MineLifeConfig(w=10, h=10,
mine_ratio=0.15))

    print("--- 脳内シミュレーション開始 (KQI Integrated) ---")

    # 10 ターン回してみる
    results = brain.tick(n=10)

    for res in results:
        kqi = res["kqi"]
        print(f"Tick {res['tick']:02d} | "
              f"G:{kqi['G']:.2f} σ:{kqi['Q']:.2f} " # Q は便宜上 sigma の逆数的な
位置づけだがここではQを表示
              f"-> Motion:{kqi['M']} "
              f"-> Depth(D):{kqi['D']:.4f} "
              f"| Tags: {res['tags']}")

```

説明

A.2 確率的論理モジュール (Improvisation_SLM.py) このモジュールは、トップロック (Toprock), フットワーク (Footwork), フリーズ (Freeze) 間の遷移確率を管理する.

```

Python
import random
import numpy as np
from enum import Enum

class State(Enum):
    F = 0 # Flow (Toprock)
    B = 1 # Burst (Power)
    S = 2 # Stillness (Freeze)

class ImprovisationSLM:

```

```

def __init__(self):
    self.state = State.S
    self.entropy_history = []

def compute_context_entropy(self, audio_features):
    """
    オーディオバッファのシャノンエントロピーを計算し、「カオス」を検出する。
    """
    # レポートの簡潔さのために簡略化:
    energy = np.mean(audio_features)
    return -energy * np.log(energy + 1e-9)

def step(self, current_state, kqi_depth):
    # 高深度 = パワー (State B) の高確率
    # 低深度 = フリーズ (State S) の高確率
    if kqi_depth > 5.0:
        return State.B
    elif kqi_depth < 0.5:
        return State.S
    else:
        return State.F

```

A.3 リアルタイム知覚器 (realtime_audio_engine.py) WASAPI ループバックと特徴量抽出を処理する。

```

Python
import numpy as np
import librosa

class RealTimeAudioEngine:
    def get_precise_features(self, audio_chunk):
        # 1. ステレオとゲインの処理
        if audio_chunk.ndim > 1:
            y = np.mean(audio_chunk, axis=1).flatten()
        else:
            y = audio_chunk.flatten()

        # 弱いラップトップマイクのためにゲインをブースト
        y = y * 15.0

        # 2. スペクトル分析
        mfcc = librosa.feature.mfcc(y=y, sr=48000, n_mfcc=20)
        chroma = librosa.feature.chroma_stft(y=y, sr=48000)
        onset = librosa.onset.onset_strength(y=y, sr=48000)

        # 3. Z スコア正規化
        # 異なる音楽ボリュームを扱うために重要
        mfcc_norm = (mfcc - np.mean(mfcc)) / (np.std(mfcc) + 1e-6)

        return np.hstack([np.mean(mfcc_norm, axis=1), np.mean(chroma, axis=1),
                           np.mean(onset)])

```

A.4 頭脳 (sasaki_brain.py) 知覚 (Perception) と行動 (Action) を結びつける中央オーケストレーター.

Python

```
class SasakiLiveBrain:
    def decide_style(self, audio_vector):
        self.t += 1
        energy = audio_vector[-1]

        # --- 心拍数の修正 ---
        # エネルギーが 0 に近い場合, 偽の「潜在意識」パルスを提供する
        if energy < 0.01:
            energy = 0.02 * (1.0 + math.sin(self.t * 0.1))

        # 論理計算
        r_val = (energy ** 2) * 50.0
        omega_val = np.var(audio_vector[:20]) * 20.0

        kqi_res = self.kqi.process(r=r_val, omega=omega_val, sigma=0.1,
                                   t=self.t, rho=0.5)

        # 潜在的意志の注入 (The Ghost)
        self.latent_will += np.random.randn(128) * 0.05
        self.latent_will = np.clip(self.latent_will, -1, 1)

        return kqi_res, self.latent_will
```

A.5 訓練ループ (train_multimodel_residual.py) 「プリズンブレイク (Prison Break)」トレーニングロジック.

Python

損失関数の簡略化されたスニペット

```
def compute_loss(real, fake, velocity, audio_energy):
    # 1. 敵対的損失
    loss_adv = criterion_GAN(discriminator(fake), True)

    # 2. 物理損失 (MSE)
    loss_phys = criterion_MSE(real, fake)

    # 3. 運動学的ペナルティ (フェーズ 7)
    # 音楽が大きい (energy > 0.5) が, 速度が低い (v < 1.0) 場合...
    kinetic_violation = torch.relu(1.0 - velocity) * (audio_energy >
0.5).float()
    loss_kinetic = torch.mean(kinetic_violation) * 10.0

    return loss_adv + loss_phys + loss_kinetic
```

11. ユーザーマニュアルと操作ガイド (User Manual and Operational Guide)

本章では, 将来の学生や研究者が **Sasaki-GAN** システムを運用するために必要なドキュメントを提供する.

11.1. システム要件 (System Requirements)

10.1.1 ハードウェア仕様 (Hardware Specifications)

- CPU: Intel Core i7-10750H 以上 (NumPy のために AVX2 サポートが必須) .
- GPU: NVIDIA RTX 3060 (6GB VRAM) 以上. > 30 FPS には RTX 4070 (8GB) を推奨.
- RAM: 16GB DDR4 (Unity Editor + Python の同時実行には 32GB を推奨) .
- Audio: WASAPI 互換サウンドカード (ループバックサポートが必須) .

10.1.2 ソフトウェア依存関係 (Software Dependencies)

- Operating System: Windows 10/11 (WASAPI に必須).
- Python: バージョン 3.10.x (Torch 2.0 互換性).
- Blender: バージョン 3.6 LTS (内部 Python 3.10).
- CUDA: Toolkit 11.8 (cuDNN 8.5).

11.2. インストールガイド (Installation Guide)

10.2.1 環境セットアップ (Environment Setup)

環境管理には conda の使用を推奨する.

```
conda create -n gcn python=3.10
conda activate gcn
pip install torch torchvision torchaudio --index-url
[https://download.pytorch.org/whl/cu118](https://download.pytorch.org/whl/cu1
18)
pip install librosa sounddevice numpy matplotlib scipy pyyaml
pip install faiss-cpu # 検索デコーダ用
```

10.2.2 データセットの準備 (Dataset Preparation)

BRACE Dataset のコンテンツ (brace_audio/ および brace_video/) をダウンロードする.

同期スクリプトを実行する:

```
Bash
python tools/synchronize_brace.py --shift_ms 1160
```

注: 1160ms のシフトは, 元のデータセットに内在するラグを修正する. トポロジー修正ツールを実行する:

```
Bash
python tools/fix_topology.py --target nturkbd120
これにより BRACE_25J_Fixed.npy が生成される.
```

11.3. システムの実行 (Running the System)

10.3.1 トレーニングモード (Training Mode) モデルをスクラッチから再訓練するには (警告: 約 72 時間かかる):

```
Bash
python train_multimodel_residual.py --config configs/sasaki_residual.yaml
```

主要なフラグ (Key Flags): `l --resume_epoch N`: チェックポイントから再開する. `l --prison_break`: 運動学的ペナルティを有効にする (エポック 20 以降で使用). `l --ghost_weight 0.1`: 潜在的意志の分散を設定する.

10.3.2 ライブパフォーマンスモード (Live Performance Mode) これがメインのインタラクシヨンモードである.

1. Blender ブリッジの開始:

- `Sasaki_Visualizer.blend` を開く.
- `Scripture Editor` に移動 → `blender_receiver.py` を開く.
- "Run Script" をクリックする. コンソールに "Listening on :5000" と表示されるはずである.

2. 頭脳 (Brain) の開始:

```
Bash
python run_sasaki_live.py --mode production
```

3. 音楽の再生:

- システム上で任意のオーディオ (Spotify/YouTube など) を再生する.
- ターミナルに **KQI ダッシュボード** (Depth, Quality, State) が表示される.

11.4. トラブルシューティング (Troubleshooting)

10.4.1 エラー: "Input Overflow" (PyAudio)

- **症状 (Symptom):** ターミナルが "Input Overflow" を吐き出し, アニメーションがカクつく.
- **原因 (Cause):** `ListenerThread` がサンプリングレート (48kHz) よりも遅くデータを処理している. これは通常, オーディオコールバック内に `print()` ステートメントが残っている場合に発生する.
- **修正 (Fix):** コールバックからすべての I/O を削除する. `AudioQ` に最大サイズ (例: 1024) を設定し, 満杯の場合はパケットを破棄するようにする.

10.4.2 エラー: "Bone Length Explosion"

- **症状 (Symptom):** キャラクターの腕が無限に伸びる.
- **原因 (Cause):** GAN が NaN (Not a Number) または Infinity 値を生成した. 通常は正規化層でのゼロ除算が原因である.
- **修正 (Fix):** `HPI_GCN` 出力に `torch.clamp(val, -10, 10)` を実装した. これが続く場合はスクリプトを再起動する. 内部 LSTM の隠れ状態が破損している可能性がある.

10.4.3 機能: "The Zombie Drift"

- **症状 (Symptom):** 10 分かけてキャラクターがゆっくりと中心から回転してずれていく.

-
- **原因 (Cause):** ルート速度の累積 ($P_t = P_{\{t-1\}} + V_t$) が浮動小数点誤差を蓄積している.
 - **修正 (Fix):** オーディオエネルギーが < 0.01 (静寂) の状態が 5 秒以上続くと, システムは自動的に「センターリセット (Center Reset)」を行う. OpenCV ウィンドウで R を押すことでリセットを強制することもできる.

12. バグ年代記（開発ログ）（The Bug Chronicles: Development Log）

本章では、Sasaki-GAN の開発中に遭遇した具体的かつ重大な障害について詳述する。これは、ニューロシンボリックなダンス生成における一般的な落とし穴に関する技術的なリファレンスとして機能する。

12.1. 「凍結したエポック」(エポック 0-10) (The "Frozen Epoch")

- **日付:** 2025 年 11 月 4 日 **症状:** 生成器 (Generator) の損失曲線が正確に 0.693 ($\ln 2$) に留まり、識別器 (Discriminator) は 0.0 に低下した。

エラーログ:

```
Plaintext
Epoch 10/50
Settings: L_adv=1.0, L_phys=0.0
Generator Loss: 0.6931 (Static)
Discriminator Loss: 0.0000 (Perfect)
WARNING: Gradient Norm for Generator is 0.00000e+00
```

- **診断:** 勾配消失。識別器が早期に完璧になりすぎた。
- **修正:** 識別器に Spectral Normalization を実装し、実際の入力に Gaussian Noise ($\sigma=0.1$) を追加した。

12.2. 「滑る足」(エポック 25) (The "Sliding Foot")

- **日付:** 2025 年 12 月 12 日
- **症状:** キャラクターがトップロックを実行したが、グローバルルート位置が左に 2 メートルずれた。

コードトレース: sasaki_brain.py 内:

```
Python
# Old Logic
root_pos += predicted_velocity * delta_t
```

ReLU 活性化が正の平均シフトを引き起こしたため、predicted_velocity は 0.001 の定数バイアスを持っていた。

- **修正:** 分布をゼロ中心にするために、LeakyReLU (負の勾配 0.2) に切り替えた。

12.3. 「メモリリーク」(ライブ実行) (The "Memory Leak")

- **日付:** 2026 年 1 月 5 日
- **症状:** 4 分間の再生後、FPS が 30 から 5 に低下した。

エラーログ:

```
Plaintext
RuntimeError: CUDA out of memory. Tried to allocate 20.00 MiB (GPU 0; 8.00 GiB
total capacity; 7.20 GiB already allocated)
```

- **診断:** Python の memory_buffer リストが、計算グラフにまだ接続されているテンソル (requires_grad=True) を追加していた。 **修正:** 履歴バッファのスナップショットを取得する前に .detach().cpu().numpy() を追加した。

12.4. 「UDP パケット損失」(Blender) (The "UDP Packet Loss")

- **日付:** 2026 年 1 月 15 日
- **症状:** Blender のキャラクターが未来へ 10 フレーム「テレポート」する。

- **診断:** Blender のモーダルオペレーターがティックするよりも速くパケットを送信していた。OS のネットワークバッファがいっぱいになり、Blender は最も古いパケットを最初に読み取っていた (FIFO)。
- **修正:** 動作を LIFO (Last In First Out) に変更した。Blender スクリプトは現在、ソケットバッファ全体をドレイン (排出) し、最後に受信したパケットのみを処理する。

12.5. 「静寂パニック」 (The "Silence Panic")

- **日付:** 2026 年 1 月 20 日
- **症状:** 音楽が止まると、GAN がランダムな高周波ノイズを出力した。
- **理由:** Z スコア正規化がゼロ除算を起こした ($\text{std_dev} = 0$)。
- **修正:** 分母に $\text{epsilon} = 1\text{e-}6$ を追加し、 $\text{RMS} < 0.001$ の場合に出力ベクトルを強制的に zeros にする「サイレンスゲート (Silence Gate)」を追加した。

[改ページ]

付録 B: 補足コードリポジトリ (Supplementary Code Repository)

B.1 データ拡張 (augment_data.py) このスクリプトは、ランダムな回転 (Y 軸) とミラーリングをデータセットに適用し、トレーニングデータサイズを効果的に 4 倍にする。

Python

```
def augment_data(data):
    # 1. ミラーリング
    mirrored = data.copy()
    mirrored[:, :, 0] *= -1 # Flip X
    # 左右の関節を入れ替える (NTU に基づくインデックス)
    mirrored = swap_joints(mirrored, left=[5,6,7], right=[9,10,11])

    # 2. 回転
    rotated = []
    for angle in [90, 180, 270]:
        r_data = rotate_y(data, angle)
        rotated.append(r_data)

    return np.concatenate([data, mirrored] + rotated)
```

B.2 COCO-17 グラフ定義 (coco_17.py) ゼロパディング・ブリッジに使用される最小限のグラフ構造。

Python

```
# Edge List for COCO-17
edges = [
    (0, 1), (0, 2), # Nose to Eyes
    (1, 3), (2, 4), # Eyes to Ears
    (5, 7), (7, 9), # Left Arm
    (6, 8), (8, 10), # Right Arm
    (11, 13), (13, 15), # Left Leg
    (12, 14), (14, 16), # Right Leg
    (5, 6), (11, 12), # Shoulder/Hip Bridges
    (5, 11), (6, 12) # Torso
]
```

B.3 3D 可視化ツール (visualize_3d.py) Matplotlib を使用したオフライン検証用。

Python

```
def plot_3d(pose, ax):
    ax.cla()
    for edge in edges:
        p1 = pose[:, edge[0]]
```

```
    p2 = pose[:, edge[1]]
    ax.plot([p1[0], p2[0]], [p1[1], p2[1]], [p1[2], p2[2]], 'b-')
ax.set_xlim(-1, 1)
ax.set_ylim(-1, 1)
ax.set_zlim(-1, 1)
```